

4.1 Basic Concepts

Introduction

CPU scheduling is the basis of multiprogrammed operating systems. By switching the CPU among processes, the operating system can make the computer more productive. In this chapter, we introduce basic CPU-scheduling concepts and present several CPU-scheduling algorithms, including real-time systems. We also consider the problem of selecting an algorithm for a particular system.

In the chapter Threads & Concurrency, we introduced threads to the process model. On modern operating systems it is kernel-level threads—not processes—that are in fact being scheduled by the operating system. However, the terms "process scheduling" and "thread scheduling" are often used interchangeably. In this chapter, we use **process scheduling** when discussing general scheduling concepts and **thread scheduling** to refer to thread-specific ideas.

Similarly, in the chapter Introduction, we describe how a **core** is the basic computational unit of a CPU, and that a process executes on a CPU's core. However, in many instances in this chapter, when we use the general terminology of scheduling a process to "run on a CPU", we are implying that the process is running on a CPU's core.

Chapter objectives

- Describe various CPU scheduling algorithms.
 - Assess CPU scheduling algorithms based on scheduling criteria.
 - Explain the issues related to multiprocessor and multicore scheduling.
 - Describe various real-time scheduling algorithms.
 - Describe the scheduling algorithms used in the Windows, Linux, and Solaris operating systems.
 - Apply modeling and simulations to evaluate CPU scheduling algorithms.
 - Design a program that implements several different CPU scheduling algorithms.
-

Basic concepts

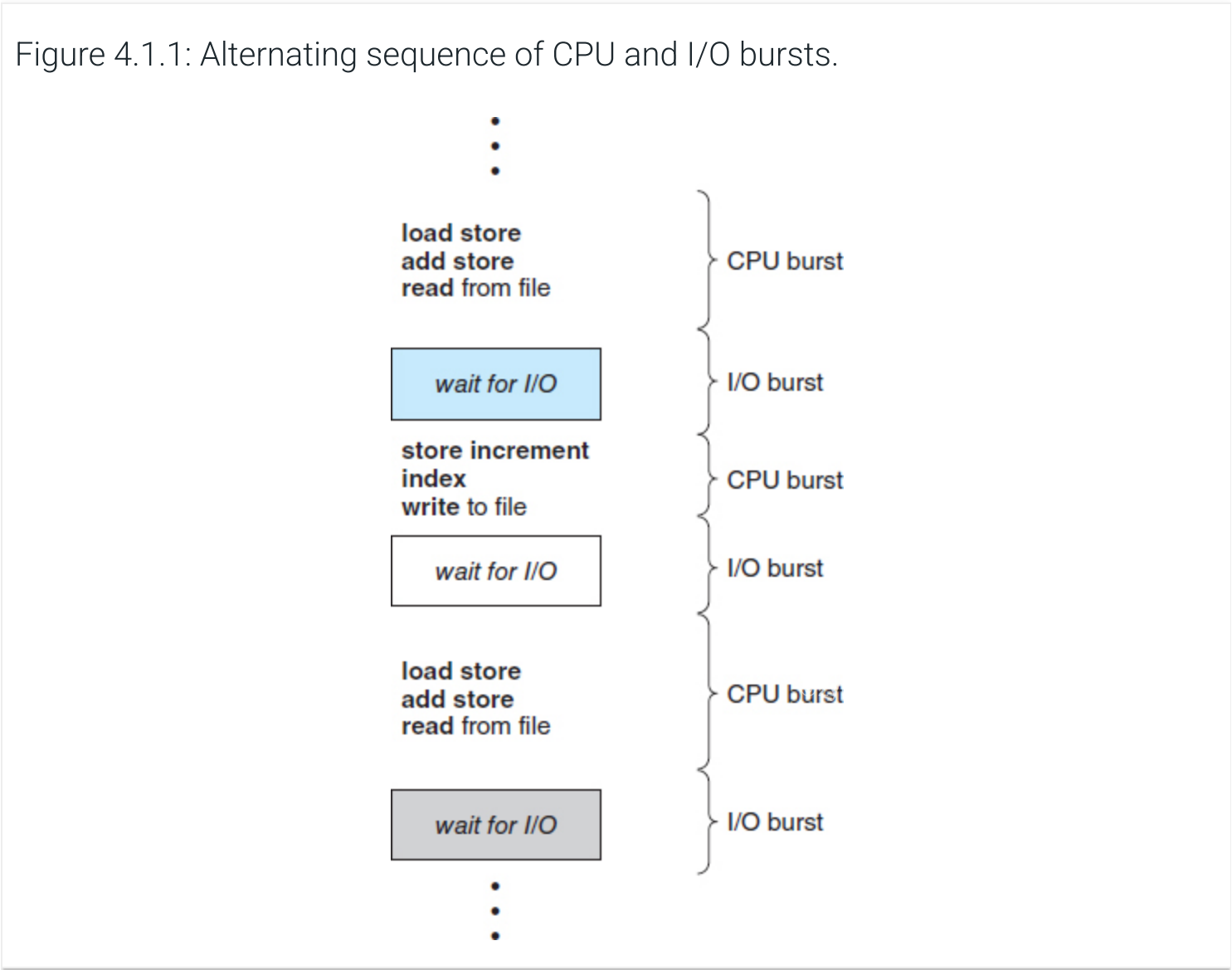
In a system with a single CPU core, only one process can run at a time. Others must wait until the CPU's core is free and can be rescheduled. The objective of multiprogramming is to have some process running at all times, to maximize CPU utilization. The idea is relatively simple. A process is executed until it must wait, typically for the completion of some I/O request. In a simple computer system, the CPU then just sits

idle. All this waiting time is wasted; no useful work is accomplished. With multiprogramming, we try to use this time productively. Several processes are kept in memory at one time. When one process has to wait, the operating system takes the CPU away from that process and gives the CPU to another process. This pattern continues. Every time one process has to wait, another process can take over use of the CPU. On a multicore system, this concept of keeping the CPU busy is extended to all processing cores on the system.

Scheduling of this kind is a fundamental operating-system function. Almost all computer resources are scheduled before use. The CPU is, of course, one of the primary computer resources. Thus, its scheduling is central to operating-system design.

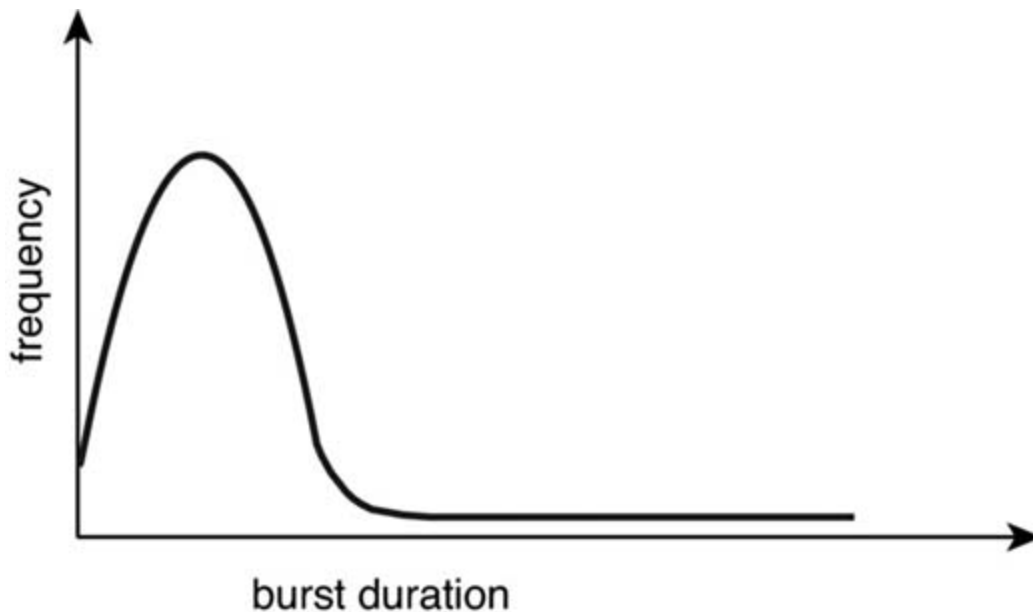
CPU-I/O burst cycle

The success of CPU scheduling depends on an observed property of processes: process execution consists of a **cycle** of CPU execution and I/O wait. Processes alternate between these two states. Process execution begins with a **CPU burst**. That is followed by an **I/O burst**, which is followed by another CPU burst, then another I/O burst, and so on. Eventually, the final CPU burst ends with a system request to terminate execution (Figure 4.1.1).



The durations of CPU bursts have been measured extensively. Although they vary greatly from process to process and from computer to computer, they tend to have a frequency curve similar to that shown in Figure 4.1.2. The curve is generally characterized as exponential or hyperexponential, with a large number of short CPU bursts and a small number of long CPU bursts. An I/O-bound program typically has many short CPU bursts. A CPU-bound program might have a few long CPU bursts. This distribution can be important when implementing a CPU-scheduling algorithm.

Figure 4.1.2: Histogram of CPU-burst durations.



CPU scheduler

Whenever the CPU becomes idle, the operating system must select one of the processes in the ready queue to be executed. The selection process is carried out by the **CPU scheduler**, which selects a process from the processes in memory that are ready to execute and allocates the CPU to that process.

Note that the ready queue is not necessarily a first-in, first-out (FIFO) queue. As we shall see when we consider the various scheduling algorithms, a ready queue can be implemented as a FIFO queue, a priority queue, a tree, or simply an unordered linked list. Conceptually, however, all the processes in the ready queue are lined up waiting for a chance to run on the CPU. The records in the queues are generally process control blocks (PCBs) of the processes.

Preemptive and nonpreemptive scheduling

CPU-scheduling decisions may take place under the following four circumstances:

1. When a process switches from the running state to the waiting state (for example, as the result of an I/O request or an invocation of `wait()` for the termination of a child process)

2. When a process switches from the running state to the ready state (for example, when an interrupt occurs)
3. When a process switches from the waiting state to the ready state (for example, at completion of I/O)
4. When a process terminates

For situations 1 and 4, there is no choice in terms of scheduling. A new process (if one exists in the ready queue) must be selected for execution. There is a choice, however, for situations 2 and 3.

When scheduling takes place only under circumstances 1 and 4, we say that the scheduling scheme is **nonpreemptive** or **cooperative**. Otherwise, it is **preemptive**. Under nonpreemptive scheduling, once the CPU has been allocated to a process, the process keeps the CPU until it releases it either by terminating or by switching to the waiting state. Virtually all modern operating systems including Windows, macOS, Linux, and UNIX use preemptive scheduling algorithms.

Unfortunately, preemptive scheduling can result in race conditions when data are shared among several processes. Consider the case of two processes that share data. While one process is updating the data, it is preempted so that the second process can run. The second process then tries to read the data, which are in an inconsistent state. This issue will be explored in detail in the chapter Synchronization Tools.

Preemption also affects the design of the operating-system kernel. During the processing of a system call, the kernel may be busy with an activity on behalf of a process. Such activities may involve changing important kernel data (for instance, I/O queues). What happens if the process is preempted in the middle of these changes and the kernel (or the device driver) needs to read or modify the same structure? Chaos ensues. As will be discussed in Section [5.2](#), operating-system kernels can be designed as either nonpreemptive or preemptive. A nonpreemptive kernel will wait for a system call to complete or for a process to block while waiting for I/O to complete to take place before doing a context switch. This scheme ensures that the kernel structure is simple, since the kernel will not preempt a process while the kernel data structures are in an inconsistent state. Unfortunately, this kernel-execution model is a poor one for supporting real-time computing, where tasks must complete execution within a given time frame. In Section [4.6](#), we explore scheduling demands of real-time systems. A preemptive kernel requires mechanisms such as mutex locks to prevent race conditions when accessing shared kernel data structures. Most modern operating systems are now fully preemptive when running in kernel mode.

Because interrupts can, by definition, occur at any time, and because they cannot always be ignored by the kernel, the sections of code affected by interrupts must be guarded from simultaneous use. The operating system needs to accept interrupts at almost all times. Otherwise, input might be lost or output overwritten. So that these sections of code are not accessed concurrently by several processes, they disable interrupts at entry and reenables interrupts at exit. It is important to note that sections of code that disable interrupts do not occur very often and typically contain few instructions.

Dispatcher

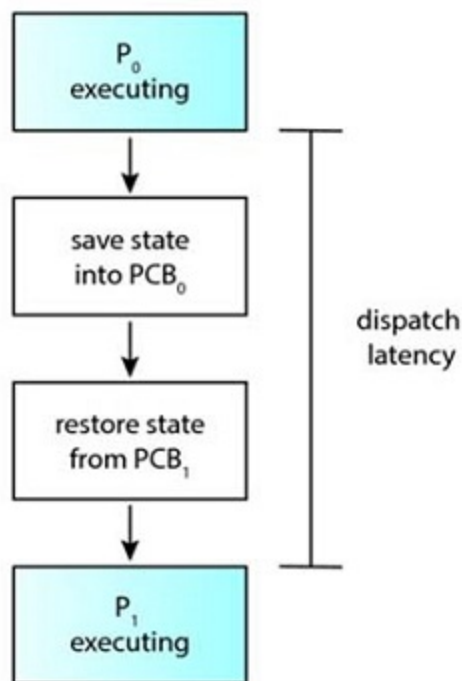
Another component involved in the CPU-scheduling function is the **dispatcher**. The dispatcher is the module that gives control of the CPU's core to the process selected by the CPU scheduler. This function

involves the following:

- Switching context from one process to another
- Switching to user mode
- Jumping to the proper location in the user program to resume that program

The dispatcher should be as fast as possible, since it is invoked during every context switch. The time it takes for the dispatcher to stop one process and start another running is known as the **dispatch latency** and is illustrated in Figure [4.1.3](#).

Figure 4.1.3: The role of the dispatcher.



An interesting question to consider is, how often do context switches occur? On a system-wide level, the number of context switches can be obtained by using the **vmstat** command that is available on Linux systems. Below is the output (which has been trimmed) from the command

```
vmstat 1 3
```

This command provides 3 lines of output over a 1-second delay:

```
-----cpu-----
24
225
339
```

The first line gives the average number of context switches over 1 second since the system booted, and the next two lines give the number of context switches over two 1-second intervals. Since this machine

booted, it has averaged 24 context switches per second. And in the past second, 339 context switches were made, with 225 context switches in the second prior to that.

We can also use the `/proc` file system to determine the number of context switches for a given process. For example, the contents of the file `/proc/2166/status` will list various statistics for the process with `pid = 2166`. The command

```
cat /proc/2166/status
```

provides the following trimmed output:

```
voluntary_ctxt_switches      150
nonvoluntary_ctxt_switches   8
```

This output shows the number of context switches over the lifetime of the process. Notice the distinction between **voluntary** and **nonvoluntary** context switches. A voluntary context switch occurs when a process has given up control of the CPU because it requires a resource that is currently unavailable (such as blocking for I/O.) A nonvoluntary context switch occurs when the CPU has been taken away from a process, such as when its time slice has expired or it has been preempted by a higher-priority process.

**PARTICIPATION
ACTIVITY**

4.1.1: Section review questions.



1) Which of the following is true of nonpreemptive scheduling?



- ☐ Nonpreemptive scheduling requires a timer.
- ☐ A process keeps the CPU until it releases the CPU either by terminating or by switching to the waiting state.
- ☐ Nonpreemptive scheduling incurs a cost associated with access to shared data.

2) Which of the following statements best describes the role of the dispatcher?

- ☐ The dispatcher is involved with making scheduling decisions.
- ☐ The dispatcher gives control of the CPU to the process selected by the scheduler.
- ☐ The dispatcher is not involved during a context switch.

Section glossary

cycle: Repeating loop

CPU burst: Scheduling process state in which the process executes on CPU.

I/O burst: Scheduling process state in which the CPU performs I/O.

CPU scheduler: Kernel routine that selects a thread from the threads that are ready to execute and allocates a core to that thread.

nonpreemptive: Under nonpreemptive scheduling, once a core has been allocated to a thread the thread keeps the core until it releases the core either by terminating or by switching to the waiting state.

cooperative: A form of scheduling in which threads voluntarily move from the running state.

preemptive: A form of scheduling in which processes or threads are involuntarily moved from the running state (by for example a timer signaling the kernel to allow the next thread to run).

dispatcher: The dispatcher is the kernel routine that gives control of a core to the thread selected by the scheduler.

dispatch latency: The time it takes for the dispatcher to stop one thread and start another running.

4.2 Scheduling criteria

Different CPU-scheduling algorithms have different properties, and the choice of a particular algorithm may favor one class of processes over another. In choosing which algorithm to use in a particular situation, we must consider the properties of the various algorithms.

Many criteria have been suggested for comparing CPU-scheduling algorithms. Which characteristics are used for comparison can make a substantial difference in which algorithm is judged to be best. The criteria include the following:

- **CPU utilization.** We want to keep the CPU as busy as possible. Conceptually, CPU utilization can range from 0 to 100 percent. In a real system, it should range from 40 percent (for a lightly loaded system) to 90 percent (for a heavily loaded system). (CPU utilization can be obtained by using the `top` command on Linux, macOS, and UNIX systems.)
- **Throughput.** If the CPU is busy executing processes, then work is being done. One measure of work is the number of processes that are completed per time unit, called **throughput**. For long processes, this rate may be one process over several seconds; for short transactions, it may be tens of processes per second.
- **Turnaround time.** From the point of view of a particular process, the important criterion is how long it takes to execute that process. The interval from the time of submission of a process to the time of completion is the turnaround time. Turnaround time is the sum of the periods spent waiting in the ready queue, executing on the CPU, and doing I/O.
- **Waiting time.** The CPU-scheduling algorithm does not affect the amount of time during which a process executes or does I/O. It affects only the amount of time that a process spends waiting in the ready queue. Waiting time is the sum of the periods spent waiting in the ready queue.
- **Response time.** In an interactive system, turnaround time may not be the best criterion. Often, a process can produce some output fairly early and can continue computing new results while previous results are being output to the user. Thus, another measure is the time from the submission of a request until the first response is produced. This measure, called response time, is the time it takes to start responding, not the time it takes to output the response.

It is desirable to maximize CPU utilization and throughput and to minimize turnaround time, waiting time, and response time. In most cases, we optimize the average measure. However, under some circumstances, we prefer to optimize the minimum or maximum values rather than the average. For example, to guarantee that all users get good service, we may want to minimize the maximum response time.

Investigators have suggested that, for interactive systems (such as a PC desktop or laptop system), it is more important to minimize the variance in the response time than to minimize the average response time. A system with reasonable and predictable response time may be considered more desirable than a system that is faster on the average but is highly variable. However, little work has been done on CPU-scheduling algorithms that minimize variance.

As we discuss various CPU-scheduling algorithms in the following section, we illustrate their operation. An accurate illustration should involve many processes, each a sequence of several hundred CPU bursts and I/O bursts. For simplicity, though, we consider only one CPU burst (in milliseconds) per process in our examples. Our measure of comparison is the average waiting time. More elaborate evaluation mechanisms are discussed in Section [Missing Reference : Algorithm evaluation](#).





_____ is the number of processes that are completed per time unit.



- ☐ CPU utilization
- ☐ Response time
- ☐ Throughput

Section glossary

throughput: Generally, the amount of work done over time. In scheduling, the number of threads completed per unit time.

4.3 Scheduling algorithms

CPU scheduling deals with the problem of deciding which of the processes in the ready queue is to be allocated the CPU's core. There are many different CPU-scheduling algorithms. In this section, we describe several of them. Although most modern CPU architectures have multiple processing cores, we describe these scheduling algorithms in the context of only one processing core available. That is, a single CPU that has a single processing core, thus the system is capable of only running one process at a time. In [Section 4.5](#) we discuss CPU scheduling in the context of multiprocessor systems.

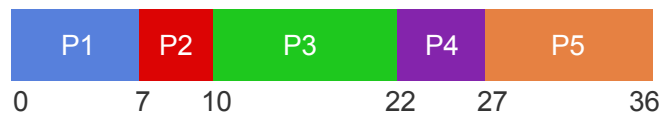
Although in practice it is not possible to know the length of a CPU burst of a process in advance as the length of a burst is dependent upon the runtime behavior of the process - such as when the process next performs I/O - we associate each process with a CPU burst time when demonstrating each of these scheduling algorithms.

First-come, first-served scheduling

By far the simplest CPU-scheduling algorithm is the **first-come, first-served (FCFS)** scheduling algorithm. With this scheme, the process that requests the CPU first is allocated the CPU first. The implementation of the FCFS policy is easily managed with a FIFO queue. When a process enters the ready queue, its PCB is linked onto the tail of the queue. When the CPU is free, it is allocated to the process at the head of the queue. The running process is then removed from the queue. The code for FCFS scheduling is simple to write and understand.



Process	Burst Time
P1	7
P2	3
P3	12
P4	5
P5	9

Gantt Chart

FCFS Scheduling

Animation content:

Step 1: FCFS scheduling for the five processes that arrived in the order P1, P2, P3, P4, and P5. Step 2: Each process is associated with a burst time. Step 3: A Gantt Chart is used to demonstrate the order in which processes are selected for scheduling using the FCFS scheduling algorithm. Step 4: Process P1 is selected first as it is the first process to arrive. It begins running at time 0. After P1 finishes running, P2 is selected to run at time 7. Step 5: Process P3 is selected next and runs at time 10. Process P4 is selected to run at time 22. Step 6: P5 arrives last, and thus it is the last process to be selected by the FCFS scheduling algorithm and begins running at time 27. At time 36, process P5 finishes running.

Animation captions:

1. FCFS scheduling for the five processes that arrived in the order P1, P2, P3, P4, and P5.
2. Each process is associated with a burst time.
3. A Gantt Chart is used to demonstrate the order in which processes are selected for scheduling using the FCFS scheduling algorithm.
4. Process P1 is selected first as it is the first process to arrive. It begins running at time 0. After P1 finishes running, P2 is selected to run at time 7.
5. Process P3 is selected next and runs at time 10. Process P4 is selected to run at time 22.
6. P5 arrives last, and thus it is the last process to be selected by the FCFS scheduling algorithm and begins running at time 27. At time 36, process P5 finishes running.

On the negative side, the average waiting time under the FCFS policy is often quite long. Consider the following set of processes that arrive at time 0, with the length of the CPU burst given in milliseconds:

Process	Burst Time
P_1	24
P_2	3
P_3	3

If the processes arrive in the order P_1, P_2, P_3 , and are served in FCFS order, we get the result shown in the following **Gantt chart**, which is a bar chart that illustrates a particular schedule, including the start and finish times of each of the participating processes:



The waiting time is 0 milliseconds for process P_1 , 24 milliseconds for process P_2 , and 27 milliseconds for process P_3 . Thus, the average waiting time is $(0 + 24 + 27) / 3 = 17$ milliseconds. If the processes arrive in the order P_2, P_3, P_1 , however, the results will be as shown in the following Gantt chart:



The average waiting time is now $(6 + 0 + 3) / 3 = 3$ milliseconds. This reduction is substantial. Thus, the average waiting time under an FCFS policy is generally not minimal and may vary substantially if the processes' CPU burst times vary greatly.

In addition, consider the performance of FCFS scheduling in a dynamic situation. Assume we have one CPU-bound process and many I/O-bound processes. As the processes flow around the system, the following scenario may result. The CPU-bound process will get and hold the CPU. During this time, all the other processes will finish their I/O and will move into the ready queue, waiting for the CPU. While the processes wait in the ready queue, the I/O devices are idle. Eventually, the CPU-bound process finishes its CPU burst and moves to an I/O device. All the I/O-bound processes, which have short CPU bursts, execute quickly and move back to the I/O queues. At this point, the CPU sits idle. The CPU-bound process will then move back to the ready queue and be allocated the CPU. Again, all the I/O processes end up waiting in the ready queue until the CPU-bound process is done. There is a **convoy effect** as all the other processes wait for the one big process to get off the CPU. This effect results in lower CPU and device utilization than might be possible if the shorter processes were allowed to go first.

Note also that the FCFS scheduling algorithm is nonpreemptive. Once the CPU has been allocated to a process, that process keeps the CPU until it releases the CPU, either by terminating or by requesting I/O. The FCFS algorithm is thus particularly troublesome for interactive systems, where it is important that each process get a share of the CPU at regular intervals. It would be disastrous to allow one process to keep the CPU for an extended period.

Shortest-job-first scheduling

A different approach to CPU scheduling is the **shortest-job-first (SJF)** scheduling algorithm. This algorithm associates with each process the length of the process's next CPU burst. When the CPU is available, it is assigned to the process that has the smallest next CPU burst. If the next CPU bursts of two processes are the same, FCFS scheduling is used to break the tie. Note that a more appropriate term for this scheduling method would be the **shortest-next-CPU-burst** algorithm, because scheduling depends on the length of the next CPU burst of a process, rather than its total length. We use the term SJF because most people and textbooks use this term to refer to this type of scheduling.

As an example of SJF scheduling, consider the following set of processes, with the length of the CPU burst given in milliseconds:

Process	Burst Time
P_1	6
P_2	8
P_3	7
P_4	3

Using SJF scheduling, we would schedule these processes according to the following Gantt chart:



PARTICIPATION
ACTIVITY

4.3.2: SJF Scheduling.

May 5, 2025, 09:40:26 -13:26:20 (GMT+00:00)

Process	Burst Time
P1	
P2	
P3	
P4	

Gantt Chart



SJF Scheduling

Animation content:

Step 1: SJF scheduling for the four processes that arrived in the order P1, P2, P3, and P4. Step 2: Each process is associated with a burst time. Step 3: A Gantt Chart is used to demonstrate the order in which processes are selected for scheduling using the SJF scheduling algorithm. Step 4: Process P4 is selected first as P4 is the process with the shortest CPU burst. P4 begins running at time 0. Process P4 finishes running at time 3. Step 5: Process P1 is selected next as P1 has the next shortest CPU burst. Process P1 begins to run at time 3 and finishes at time 9. Process P3 then starts at time 9 and ends at time 16. Step 6: Process P2 has the longest CPU burst, therefore P2 is scheduled last and runs at time 16. Process P2 finishes running at time 24.

Animation captions:

1. SJF scheduling for the four processes that arrived in the order P1, P2, P3, and P4.
2. Each process is associated with a burst time.
3. A Gantt Chart is used to demonstrate the order in which processes are selected for scheduling using the SJF scheduling algorithm.
4. Process P4 is selected first as P4 is the process with the shortest CPU burst. P4 begins running at time 0. Process P4 finishes running at time 3.
5. Process P1 is selected next as P1 has the next shortest CPU burst. Process P1 begins to run at time 3 and finishes at time 9. Process P3 then starts at time 9 and ends at time 16.
6. Process P2 has the longest CPU burst, therefore P2 is scheduled last and runs at time 16. Process P2 finishes running at time 24.

The waiting time is 3 milliseconds for process P_1 , 16 milliseconds for process P_2 , 9 milliseconds for process P_3 , and 0 milliseconds for process P_4 . Thus, the average waiting time is $(3 + 16 + 9 + 0) / 4 = 7$ milliseconds. By comparison, if we were using the FCFS scheduling scheme, the average waiting time would be 10.25 milliseconds.

The SJF scheduling algorithm is provably optimal, in that it gives the minimum average waiting time for a given set of processes. Moving a short process before a long one decreases the waiting time of the short

process more than it increases the waiting time of the long process. Consequently, the average waiting time decreases.

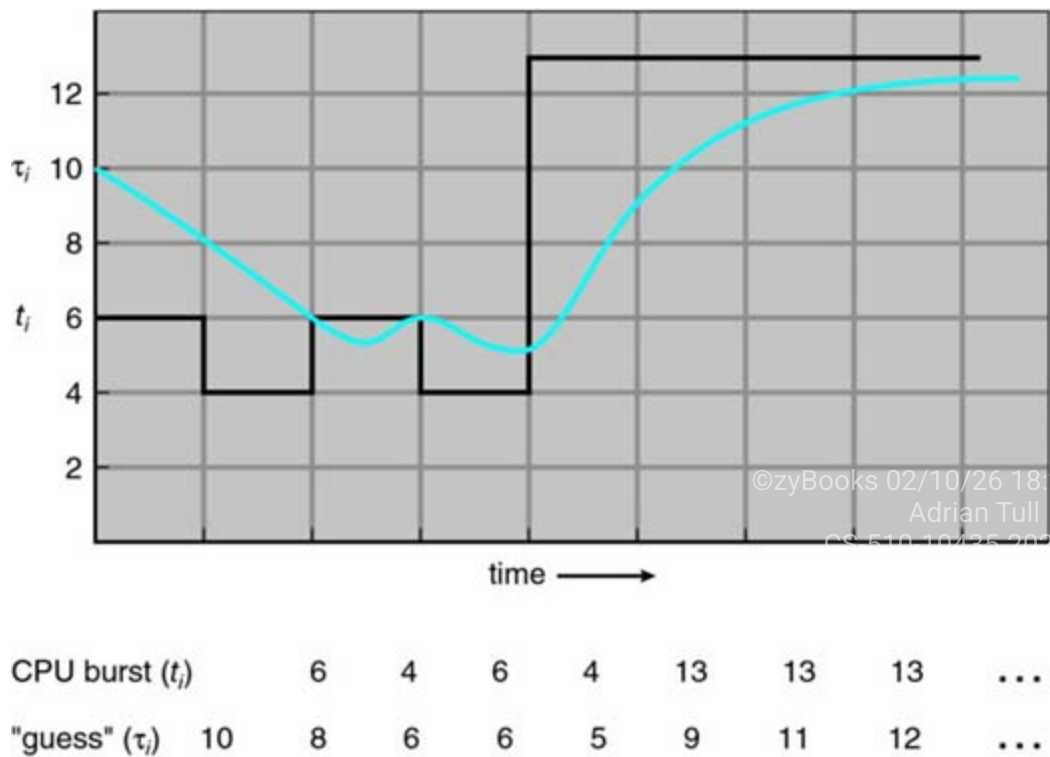
Although the SJF algorithm is optimal, it cannot be implemented at the level of CPU scheduling, as there is no way to know the length of the next CPU burst. One approach to this problem is to try to approximate SJF scheduling. We may not know the length of the next CPU burst, but we may be able to predict its value. We expect that the next CPU burst will be similar in length to the previous ones. By computing an approximation of the length of the next CPU burst, we can pick the process with the shortest predicted CPU burst.

The next CPU burst is generally predicted as an **exponential average** of the measured lengths of previous CPU bursts. We can define the exponential average with the following formula. Let t_n be the length of the n th CPU burst, and let τ_{n+1} be our predicted value for the next CPU burst. Then, for α , $0 \leq \alpha \leq 1$, define

$$\tau_{n+1} = \alpha t_n + (1 - \alpha) \tau_n.$$

The value of t_n contains our most recent information, while τ_n stores the past history. The parameter α controls the relative weight of recent and past history in our prediction. If $\alpha = 0$, then $\tau_{n+1} = \tau_n$, and recent history has no effect (current conditions are assumed to be transient). If $\alpha = 1$, then $\tau_{n+1} = t_n$, and only the most recent CPU burst matters (history is assumed to be old and irrelevant). More commonly, $\alpha = 1/2$, so recent history and past history are equally weighted. The initial τ_0 can be defined as a constant or as an overall system average. Figure 4.3.1 shows an exponential average with $\alpha = 1/2$ and $\tau_0 = 10$.

Figure 4.3.1: Prediction of the length of the next CPU burst.



To understand the behavior of the exponential average, we can expand the formula for τ_{n+1} by substituting for τ_n to find

$$\tau_{n+1} = \alpha t_n + (1 - \alpha) \alpha t_{n-1} + \cdots + (1 - \alpha)^j \alpha t_{n-j} + \cdots + (1 - \alpha)^{n+1} \tau_0.$$

Typically, α is less than 1. As a result, $(1 - \alpha)$ is also less than 1, and each successive term has less weight than its predecessor.

The SJF algorithm can be either preemptive or nonpreemptive. The choice arises when a new process arrives at the ready queue while a previous process is still executing. The next CPU burst of the newly arrived process may be shorter than what is left of the currently executing process. A preemptive SJF algorithm will preempt the currently executing process, whereas a nonpreemptive SJF algorithm will allow the currently running process to finish its CPU burst. Preemptive SJF scheduling is sometimes called **shortest-remaining-time-first** scheduling.

As an example, consider the following four processes, with the length of the CPU burst given in milliseconds:

Process	Arrival Time	Burst Time
P_1	0	8
P_2	1	4
P_3	2	9
P_4	3	5

If the processes arrive at the ready queue at the times shown and need the indicated burst times, then the resulting preemptive SJF schedule is as depicted in the following Gantt chart:



Process P_1 is started at time 0, since it is the only process in the queue. Process P_2 arrives at time 1. The remaining time for process P_1 (7 milliseconds) is larger than the time required by process P_2 (4 milliseconds), so process P_1 is preempted, and process P_2 is scheduled. The average waiting time for this example is $[(10 - 1) + (1 - 1) + (17 - 2) + (5 - 3)] / 4 = 26 / 4 = 6.5$ milliseconds. Nonpreemptive SJF scheduling would result in an average waiting time of 7.75 milliseconds.

Round-robin scheduling

The **round-robin** (RR) scheduling algorithm is similar to FCFS scheduling, but preemption is added to enable the system to switch between processes. A small unit of time, called a **time quantum** or **time slice**,

is defined. A time quantum is generally from 10 to 100 milliseconds in length. The ready queue is treated as a circular queue. The CPU scheduler goes around the ready queue, allocating the CPU to each process for a time interval of up to 1 time quantum.

To implement RR scheduling, we again treat the ready queue as a FIFO queue of processes. New processes are added to the tail of the ready queue. The CPU scheduler picks the first process from the ready queue, sets a timer to interrupt after 1 time quantum, and dispatches the process.

One of two things will then happen. The process may have a CPU burst of less than 1 time quantum. In this case, the process itself will release the CPU voluntarily. The scheduler will then proceed to the next process in the ready queue. If the CPU burst of the currently running process is longer than 1 time quantum, the timer will go off and will cause an interrupt to the operating system. A context switch will be executed, and the process will be put at the tail of the ready queue. The CPU scheduler will then select the next process in the ready queue.

PARTICIPATION
ACTIVITY

4.3.3: RR Scheduling.

Process	Burst Time
P1	0 2 5
P2	0 3
P3	0 2 5 8
P4	0 3 6 9

Time Quantum: 3

Gantt Chart



FCFS Scheduling

Animation content:

Step 1: RR scheduling for the four processes that arrived in the order P1, P2, P3, and P4. Step 2: Each process is associated with a burst time. The time quantum is the amount of time a process can run before it is preempted and the CPU is assigned to another process. Step 3: A Gantt Chart is used to demonstrate the order in which processes are selected for scheduling using the priority scheduling algorithm. Step 4: Process P1 is selected first as it is the first process to arrive. P1 begins running at time 0 where it will run for the duration of a time quantum — three time units. Step 5: Process P2 will

run next, beginning at time 3. Process P2 runs for the duration of a time quantum. Process P2 completes its CPU burst. Step 6: Process P3 and P4 run next. Both run for the duration of the time quantum. Step 7: Process P1 only runs for the remaining duration of its CPU burst. Step 8: The RR scheduling algorithm continues running until all remaining processes have completed their CPU burst.

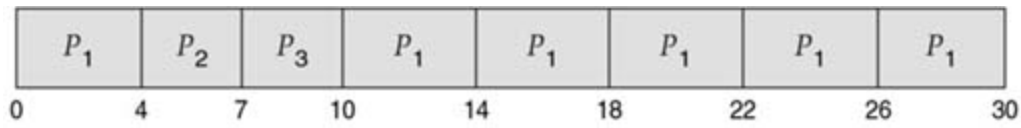
Animation captions:

1. RR scheduling for the four processes that arrived in the order P1, P2, P3, and P4.
2. Each process is associated with a burst time. The time quantum is the amount of time a process can run before it is preempted and the CPU is assigned to another process.
3. A Gantt Chart is used to demonstrate the order in which processes are selected for scheduling using the priority scheduling algorithm.
4. Process P1 is selected first as it is the first process to arrive. P1 begins running at time 0 where it will run for the duration of a time quantum – three time units.
5. Process P2 will run next, beginning at time 3. Process P2 runs for the duration of a time quantum. Process P2 completes its CPU burst.
6. Process P3 and P4 run next. Both run for the duration of the time quantum.
7. Process P1 only runs for the remaining duration of its CPU burst.
8. The RR scheduling algorithm continues running until all remaining processes have completed their CPU burst.

The average waiting time under the RR policy is often long. Consider the following set of processes that arrive at time 0, with the length of the CPU burst given in milliseconds:

Process	Burst Time
P_1	24
P_2	3
P_3	3

If we use a time quantum of 4 milliseconds, then process P_1 gets the first 4 milliseconds. Since it requires another 20 milliseconds, it is preempted after the first time quantum, and the CPU is given to the next process in the queue, process P_2 . Process P_2 does not need 4 milliseconds, so it quits before its time quantum expires. The CPU is then given to the next process, process P_3 . Once each process has received 1 time quantum, the CPU is returned to process P_1 for an additional time quantum. The resulting RR schedule is as follows:

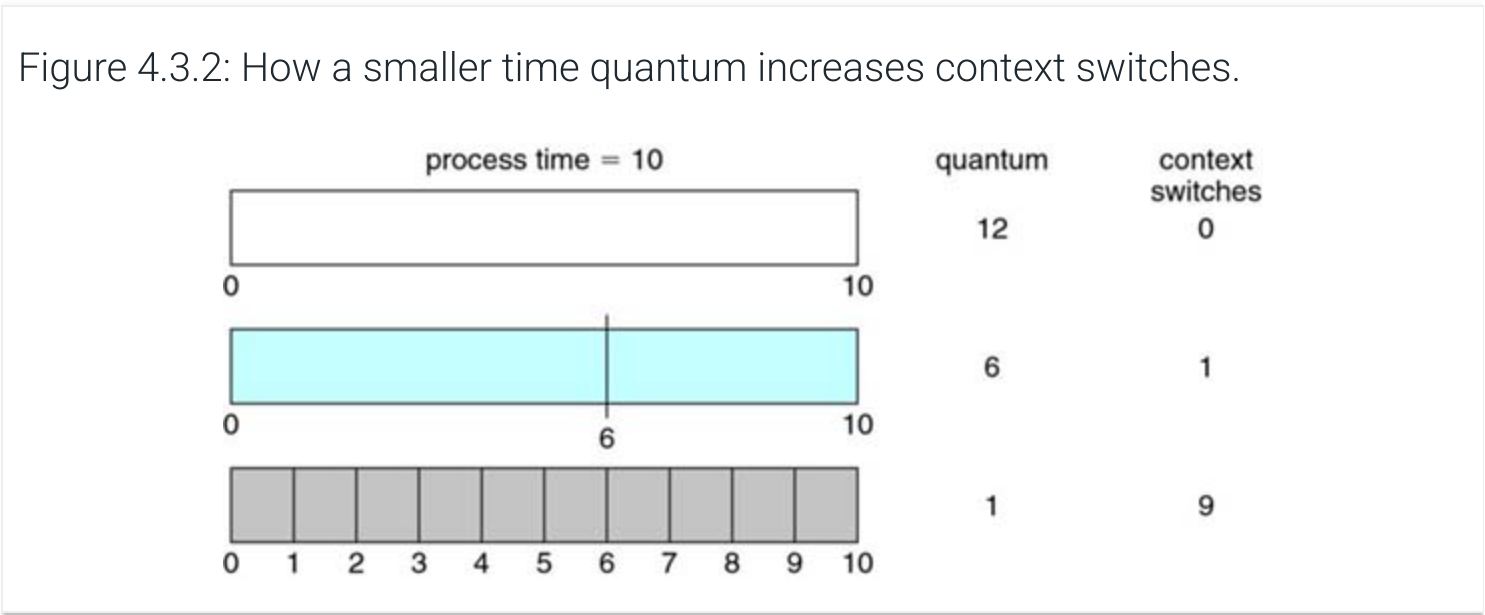


Let's calculate the average waiting time for this schedule. P_1 waits for 6 milliseconds ($10 - 4$), P_2 waits for 4 milliseconds, and P_3 waits for 7 milliseconds. Thus, the average waiting time is $17 / 3 = 5.66$ milliseconds.

In the RR scheduling algorithm, no process is allocated the CPU for more than 1 time quantum in a row (unless it is the only runnable process). If a process's CPU burst exceeds 1 time quantum, that process is preempted and is put back in the ready queue. The RR scheduling algorithm is thus preemptive.

If there are n processes in the ready queue and the time quantum is q , then each process gets $1/n$ of the CPU time in chunks of at most q time units. Each process must wait no longer than $(n - 1) \times q$ time units until its next time quantum. For example, with five processes and a time quantum of 20 milliseconds, each process will get up to 20 milliseconds every 100 milliseconds.

The performance of the RR algorithm depends heavily on the size of the time quantum. At one extreme, if the time quantum is extremely large, the RR policy is the same as the FCFS policy. In contrast, if the time quantum is extremely small (say, 1 millisecond), the RR approach can result in a large number of context switches. Assume, for example, that we have only one process of 10 time units. If the quantum is 12 time units, the process finishes in less than 1 time quantum, with no overhead. If the quantum is 6 time units, however, the process requires 2 quanta, resulting in a context switch. If the time quantum is 1 time unit, then nine context switches will occur, slowing the execution of the process accordingly Figure [4.3.2](#).

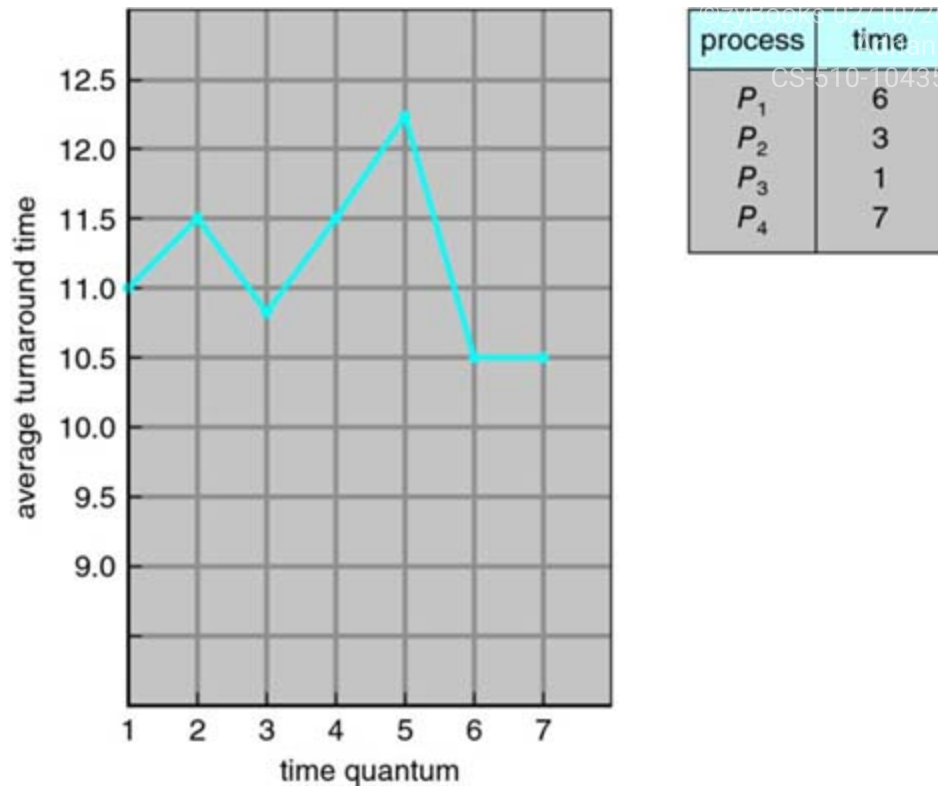


Thus, we want the time quantum to be large with respect to the context-switch time. If the context-switch time is approximately 10 percent of the time quantum, then about 10 percent of the CPU time will be spent in context switching. In practice, most modern systems have time quanta ranging from 10 to 100 milliseconds. The time required for a context switch is typically less than 10 microseconds; thus, the context-switch time is a small fraction of the time quantum.

Turnaround time also depends on the size of the time quantum. As we can see from Figure [4.3.3](#), the average turnaround time of a set of processes does not necessarily improve as the time-quantum size increases. In general, the average turnaround time can be improved if most processes finish their next CPU burst in a single time quantum. For example, given three processes of 10 time units each and a

quantum of 1 time unit, the average turnaround time is 29. If the time quantum is 10, however, the average turnaround time drops to 20. If context-switch time is added in, the average turnaround time increases even more for a smaller time quantum, since more context switches are required.

Figure 4.3.3: How turnaround time varies with the time quantum.



Although the time quantum should be large compared with the context-switch time, it should not be too large. As we pointed out earlier, if the time quantum is too large, RR scheduling degenerates to an FCFS policy. A rule of thumb is that 80 percent of the CPU bursts should be shorter than the time quantum.

Priority scheduling

The SJF algorithm is a special case of the general **priority-scheduling** algorithm. A priority is associated with each process, and the CPU is allocated to the process with the highest priority. Equal-priority processes are scheduled in FCFS order. An SJF algorithm is simply a priority algorithm where the priority (p) is the inverse of the (predicted) next CPU burst. The larger the CPU burst, the lower the priority, and vice versa.

Note that we discuss scheduling in terms of **high** priority and **low** priority. Priorities are generally indicated by some fixed range of numbers, such as 0 to 7 or 0 to 4,095. However, there is no general agreement on whether 0 is the highest or lowest priority. Some systems use low numbers to represent low priority; others use low numbers for high priority. This difference can lead to confusion. In this text, we assume that low numbers represent high priority.

As an example, consider the following set of processes, assumed to have arrived at time 0 in the order $P_1, P_2, \dots, P_5$, with the length of the CPU burst given in milliseconds:

Process	Burst Time	Priority
P_1	10	3
P_2	1	1
P_3	2	4
P_4	1	5
P_5	5	2

Using priority scheduling, we would schedule these processes according to the following Gantt chart:



The average waiting time is 8.2 milliseconds.

PARTICIPATION
ACTIVITY

4.3.4: Priority Scheduling.



Process	Burst Time	Priority
P1	10	3
P2	1	1
P3	2	4
P4	1	5

Gantt Chart



Animation content:

Static figure: A table of processes, their corresponding burst times, and their priorities is shown depicting priority scheduling. The table reads as the following:

Process	Burst Time	Priority
P1	9	3
P2	1	1
P3	5	2
P4	4	4

The burst time and priority values are crossed out.

The Gantt chart shows the processes as they are scheduled. P2 starts at time 0 and runs until time 1. P3 starts at time 1 and runs until time 6. P1 starts at time 6 and runs until time 16. P4 starts at time 16 and runs until time 17.

Animation captions:

1. Priority scheduling for the four processes P1, P2, P3, and P4.
2. Each process is associated with a burst time. Each process is also associated with a priority where a lower value indicates a higher relative priority.
3. A Gantt Chart is used to demonstrate the order in which processes are selected for scheduling using the priority scheduling algorithm.
4. Process P2 is selected first as P2 is the process with the highest priority. P2 begins running at time 0. Process P2 finishes running at time 1.
5. Process P3 is selected next as P3 has the next highest priority. Process P3 begins to run at time 1 and finishes at time 6. Process P1 then starts at time 6 and ends at time 15.
6. Process P4 the lowest priority, therefore P4 is scheduled last and runs at time 15. Process P4 finishes running at time 16.

Priorities can be defined either internally or externally. Internally defined priorities use some measurable quantity or quantities to compute the priority of a process. For example, time limits, memory requirements, the number of open files, and the ratio of average I/O burst to average CPU burst have been used in computing priorities. External priorities are set by criteria outside the operating system, such as the importance of the process, the type and amount of funds being paid for computer use, the department sponsoring the work, and other, often political, factors.

Priority scheduling can be either preemptive or nonpreemptive. When a process arrives at the ready queue, its priority is compared with the priority of the currently running process. A preemptive priority scheduling algorithm will preempt the CPU if the priority of the newly arrived process is higher than the priority of the currently running process. A nonpreemptive priority scheduling algorithm will simply put the new process at the head of the ready queue.

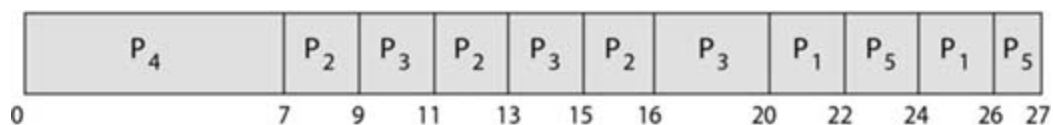
A major problem with priority scheduling algorithms is **indefinite blocking**, or **starvation**. A process that is ready to run but waiting for the CPU can be considered blocked. A priority scheduling algorithm can leave some low-priority processes waiting indefinitely. In a heavily loaded computer system, a steady stream of higher-priority processes can prevent a low-priority process from ever getting the CPU. Generally, one of two things will happen. Either the process will eventually be run (at 2 A.M. Sunday, when the system is finally lightly loaded), or the computer system will eventually crash and lose all unfinished low-priority processes. (Rumor has it that when they shut down the IBM 7094 at MIT in 1973, they found a low-priority process that had been submitted in 1967 and had not yet been run.)

A solution to the problem of indefinite blockage of low-priority processes is **aging**. Aging involves gradually increasing the priority of processes that wait in the system for a long time. For example, if priorities range from 127 (low) to 0 (high), we could periodically (say, every second) increase the priority of a waiting process by 1. Eventually, even a process with an initial priority of 127 would have the highest priority in the system and would be executed. In fact, it would take a little over 2 minutes for a priority-127 process to age to a priority-0 process.

Another option is to combine round-robin and priority scheduling in such a way that the system executes the highest-priority process and runs processes with the same priority using round-robin scheduling. Let's illustrate with an example using the following set of processes, with the burst time in milliseconds:

Process	Burst Time	Priority
P_1	4	3
P_2	5	2
P_3	8	2
P_4	7	1
P_5	3	3

Using priority scheduling with round-robin for processes with equal priority, we would schedule these processes according to the following Gantt chart using a time quantum of 2 milliseconds:

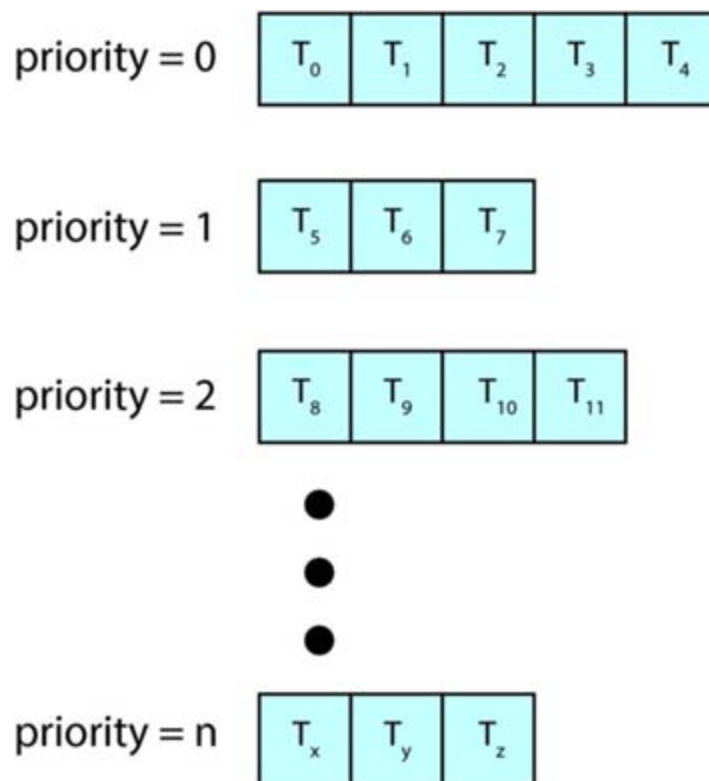


In this example, process P_4 has the highest priority, so it will run to completion. Processes P_2 and P_3 have the next-highest priority, and they will execute in a round-robin fashion. Notice that when process P_2 finishes at time 16, process P_3 is the highest-priority process, so it will run until it completes execution. Now, only processes P_1 and P_5 remain, and as they have equal priority, they will execute in round-robin order until they complete.

Multilevel queue scheduling

With both priority and round-robin scheduling, all processes may be placed in a single queue, and the scheduler then selects the process with the highest priority to run. Depending on how the queues are managed, an $O(n)$ search may be necessary to determine the highest-priority process. In practice, it is often easier to have separate queues for each distinct priority, and priority scheduling simply schedules the process in the highest-priority queue. This is illustrated in Figure 4.3.4. This approach—known as **multilevel queue**—also works well when priority scheduling is combined with round-robin: if there are multiple processes in the highest-priority queue, they are executed in round-robin order. In the most generalized form of this approach, a priority is assigned statically to each process, and a process remains in the same queue for the duration of its runtime.

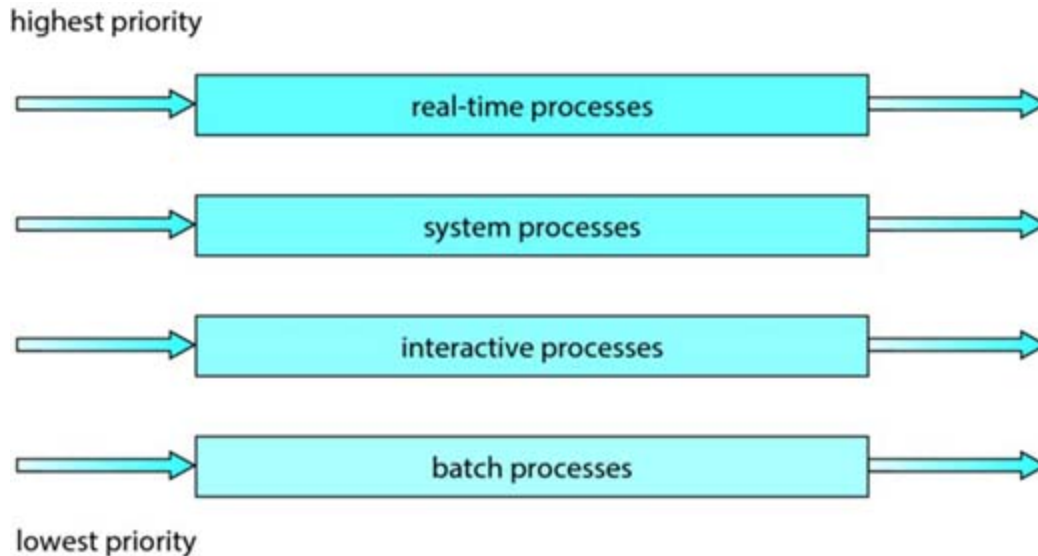
Figure 4.3.4: Separate queues for each priority.



A multilevel queue scheduling algorithm can also be used to partition processes into several separate queues based on the process type Figure 4.3.5. For example, a common division is made between **foreground** (interactive) processes and **background** (batch) processes. These two types of processes have

different response-time requirements and so may have different scheduling needs. In addition, foreground processes may have priority (externally defined) over background processes. Separate queues might be used for foreground and background processes, and each queue might have its own scheduling algorithm. The foreground queue might be scheduled by an RR algorithm, for example, while the background queue is scheduled by an FCFS algorithm.

Figure 4.3.5: Multilevel queue scheduling.



In addition, there must be scheduling among the queues, which is commonly implemented as fixed-priority preemptive scheduling. For example, the real-time queue may have absolute priority over the interactive queue.

Let's look at an example of a multilevel queue scheduling algorithm with four queues, listed below in order of priority:

1. Real-time processes
2. System processes
3. Interactive processes
4. Batch processes

Each queue has absolute priority over lower-priority queues. No process in the batch queue, for example, could run unless the queues for real-time processes, system processes, and interactive processes were all empty. If an interactive process entered the ready queue while a batch process was running, the batch process would be preempted.

Another possibility is to time-slice among the queues. Here, each queue gets a certain portion of the CPU time, which it can then schedule among its various processes. For instance, in the foreground-background queue example, the foreground queue can be given 80 percent of the CPU time for RR scheduling among its processes, while the background queue receives 20 percent of the CPU to give to its processes on an FCFS basis.

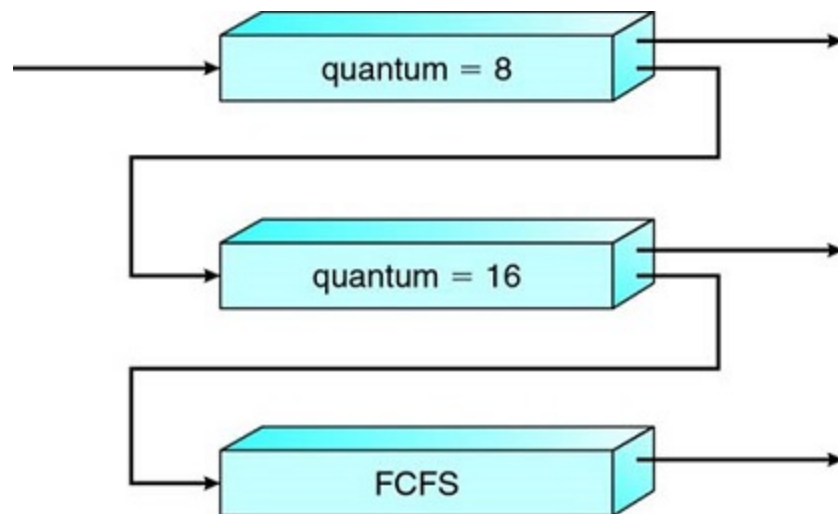
Multilevel feedback queue scheduling

Normally, when the multilevel queue scheduling algorithm is used, processes are permanently assigned to a queue when they enter the system. If there are separate queues for foreground and background processes, for example, processes do not move from one queue to the other, since processes do not change their foreground or background nature. This setup has the advantage of low scheduling overhead, but it is inflexible.

The **multilevel feedback queue** scheduling algorithm, in contrast, allows a process to move between queues. The idea is to separate processes according to the characteristics of their CPU bursts. If a process uses too much CPU time, it will be moved to a lower-priority queue. This scheme leaves I/O-bound and interactive processes—which are typically characterized by short CPU bursts—in the higher-priority queues. In addition, a process that waits too long in a lower-priority queue may be moved to a higher-priority queue. This form of aging prevents starvation.

For example, consider a multilevel feedback queue scheduler with three queues, numbered from 0 to 2. Figure 4.3.6. The scheduler first executes all processes in queue 0. Only when queue 0 is empty will it execute processes in queue 1. Similarly, processes in queue 2 will be executed only if queues 0 and 1 are empty. A process that arrives for queue 1 will preempt a process in queue 2. A process in queue 1 will in turn be preempted by a process arriving for queue 0.

Figure 4.3.6: Multilevel feedback queues.



An entering process is put in queue 0. A process in queue 0 is given a time quantum of 8 milliseconds. If it does not finish within this time, it is moved to the tail of queue 1. If queue 0 is empty, the process at the head of queue 1 is given a quantum of 16 milliseconds. If it does not complete, it is preempted and is put into queue 2. Processes in queue 2 are run on an FCFS basis but are run only when queues 0 and 1 are empty. To prevent starvation, a process that waits too long in a lower-priority queue may gradually be moved to a higher-priority queue.

This scheduling algorithm gives highest priority to any process with a CPU burst of 8 milliseconds or less. Such a process will quickly get the CPU, finish its CPU burst, and go off to its next I/O burst. Processes that need more than 8 but less than 24 milliseconds are also served quickly, although with lower priority than shorter processes. Long processes automatically sink to queue 2 and are served in FCFS order with any CPU cycles left over from queues 0 and 1.

In general, a multilevel feedback queue scheduler is defined by the following parameters:

- The number of queues
- The scheduling algorithm for each queue
- The method used to determine when to upgrade a process to a higher-priority queue
- The method used to determine when to demote a process to a lower-priority queue
- The method used to determine which queue a process will enter when that process needs service

The definition of a multilevel feedback queue scheduler makes it the most general CPU-scheduling algorithm. It can be configured to match a specific system under design. Unfortunately, it is also the most complex algorithm, since defining the best scheduler requires some means by which to select values for all the parameters.

**PARTICIPATION
ACTIVITY**

4.3.5: Section review questions.



1) In ____ scheduling, the next CPU burst is predicted with an exponential average of the measured lengths of previous CPU bursts.



- ☐ Multilevel queue
- ☐ FCFS
- ☐ SJF

2) ____ scheduling algorithm preempts the currently-running process after its time quantum has expired.



- ☐ FCFS
- ☐ RR
- ☐ Multilevel queue

3) Which of the following is true of multilevel queue scheduling?



- ☐ Processes can move among queues.
- ☐ Each queue is assigned a specific scheduling algorithm.
- ☐ Threads in one queue cannot have absolute priority over threads in lower-priority queues.

Section glossary

First-come first-served (FCFS): The simplest scheduling algorithm - the thread that requests a core first is allocated the core first, and others following get cores in the order of their requests.

Gantt chart: A bar chart that is used in the text to illustrate a schedule.

convoy effect: A scheduling phenomenon in which threads wait for the one thread to get off a core, causing overall device and CPU utilization to be suboptimal.

shortest-job-first (SJF): A scheduling algorithm that associates with each thread the length of the threads next CPU burst and schedules the shortest first.

exponential average: A calculation used in scheduling to estimate the next CPU burst time based on the previous burst times (with exponential decay on older values).

shortest-remaining-time-first (SJRF): Similar to SJF, this scheduling algorithm optimizes for the shortest remaining time until thread completion.

round-robin (RR): A scheduling algorithm that is designed especially for time-sharing systems - similar to FCFS scheduling, but preemption is added to enable the system to switch between threads.

time quantum: A small unit of time used by scheduling algorithms as a basis for determining when to preempt a thread from the CPU to allow another to run.

time slice: See time quantum.

priority-scheduling: A scheduling algorithm in which a priority is associated with each thread and the free CPU core is allocated to the thread with the highest priority.

infinite blocking: See starvation.

starvation: A scheduling risk in which a thread that is ready to run never gets put onto the CPU due to the scheduling algorithm - it is starved for CPU time.

aging: Aging is a solution to scheduling starvation and involves gradually increasing the priority of threads as they wait for CPU time.

multilevel queue: A multilevel queue scheduling algorithm partitions the ready queue into several separate queues.

foreground: A thread that is interactive and has input directed to it (such as a window currently selected as active or a terminal window that is currently selected to receive input).

background: A thread that is not currently interactive (has no interactive input directed to it) such as one in a batch job or not currently being used by a user.

multilevel feedback queue: The multilevel feedback queue scheduling algorithm that allows a process to move between queues.

4.4 Thread scheduling

In the chapter *Threads & Concurrency*, we introduced threads to the process model, distinguishing between **user-level** and **kernel-level** threads. On most modern operating systems it is kernel-level threads—not processes—that are being scheduled by the operating system. User-level threads are managed by a thread library, and the kernel is unaware of them. To run on a CPU, user-level threads must ultimately be mapped to an associated kernel-level thread, although this mapping may be indirect and may use a lightweight process (LWP). In this section, we explore scheduling issues involving user-level and kernel-level threads and offer specific examples of scheduling for Pthreads.

Contention scope

One distinction between user-level and kernel-level threads lies in how they are scheduled. On systems implementing the many-to-one (Section *Many-to-one model*) and many-to-many (Section *Many-to-many model*) models, the thread library schedules user-level threads to run on an available LWP. This scheme is known as **process-contention scope (PCS)**, since competition for the CPU takes place among threads belonging to the same process. (When we say the thread library *schedules* user threads onto available LWPs, we do not mean that the threads are actually *running* on a CPU as that further requires the operating system to schedule the LWP's kernel thread onto a physical CPU core.) To decide which kernel-level thread to schedule onto a CPU, the kernel uses **system-contention scope (SCS)**. Competition for the CPU with SCS scheduling takes place among all threads in the system. Systems using the one-to-one model (Section *One-to-one model*), such as Windows and Linux schedule threads using only SCS.

Typically, PCS is done according to priority—the scheduler selects the runnable thread with the highest priority to run. User-level thread priorities are set by the programmer and are not adjusted by the thread library, although some thread libraries may allow the programmer to change the priority of a thread. It is important to note that PCS will typically preempt the thread currently running in favor of a higher-priority

thread; however, there is no guarantee of time slicing (Section Round-robin scheduling) among threads of equal priority.

Pthread scheduling

We provided a sample POSIX Pthread program in Section Pthreads, along with an introduction to thread creation with Pthreads. Now, we highlight the POSIX Pthread API that allows specifying PCS or SCS during thread creation. Pthreads identifies the following contention scope values:

- **PTHREAD_SCOPE_PROCESS** schedules threads using PCS scheduling.
- **PTHREAD_SCOPE_SYSTEM** schedules threads using SCS scheduling.

On systems implementing the many-to-many model, the **PTHREAD_SCOPE_PROCESS** policy schedules user-level threads onto available LWPs. The number of LWPs is maintained by the thread library, perhaps using scheduler activations (Section Scheduler activations). The **PTHREAD_SCOPE_SYSTEM** scheduling policy will create and bind an LWP for each user-level thread on many-to-many systems, effectively mapping threads using the one-to-one policy.

The Pthread IPC (Interprocess Communication) provides two functions for setting—and getting—the contention scope policy:

- `pthread_attr_setscope(pthread_attr_t *attr, int scope)`
- `pthread_attr_getscope(pthread_attr_t *attr, int *scope)`

The first parameter for both functions contains a pointer to the attribute set for the thread. The second parameter for the `pthread_attr_setscope()` function is passed either the **PTHREAD_SCOPE_SYSTEM** or the **PTHREAD_SCOPE_PROCESS** value, indicating how the contention scope is to be set. In the case of `pthread_attr_getscope()`, this second parameter contains a pointer to an `int` value that is set to the current value of the contention scope. If an error occurs, each of these functions returns a nonzero value.

In Figure 4.4.1, we illustrate a Pthread scheduling API. The program first determines the existing contention scope and sets it to **PTHREAD_SCOPE_SYSTEM**. It then creates five separate threads that will run using the SCS scheduling policy. Note that on some systems, only certain contention scope values are allowed. For example, Linux and macOS systems allow only **PTHREAD_SCOPE_SYSTEM**.

Figure 4.4.1: Pthread scheduling API.

```
#include <pthread.h>
#include <stdio.h>
#define NUM_THREADS 5

int main(int argc, char *argv[])
{
    int i, scope;
    pthread_t tid[NUM_THREADS];
    pthread_attr_t attr;
```

CS-510-10435-202617-1

```

/* get the default attributes */
pthread_attr_init(&attr);

/* first inquire on the current scope */
if (pthread_attr_getscope(&attr, &scope) != 0)
    fprintf(stderr, "Unable to get scheduling scope\n");
else {
    if (scope == PTHREAD_SCOPE_PROCESS)
        printf("PTHREAD_SCOPE_PROCESS");
    else if (scope == PTHREAD_SCOPE_SYSTEM)
        printf("PTHREAD_SCOPE_SYSTEM");
    else
        fprintf(stderr, "Illegal scope value.\n");
}
/* set the scheduling algorithm to PCS or SCS */
pthread_attr_setscope(&attr, PTHREAD_SCOPE_SYSTEM);

/* create the threads */
for (i = 0; i < NUM_THREADS; i++)
    pthread_create(&tid[i], &attr, runner, NULL);

/* now join on each thread */
for (i = 0; i < NUM_THREADS; i++)
    pthread_join(tid[i], NULL);
}

/* Each thread will begin control in this function */
void *runner(void *param)
{
    /* do some work ... */
    pthread_exit(0);
}

```

OzyBooks 02/10/26 18:26 2150642
 Adrian Tull
 CS-510-10435.202617-1

PARTICIPATION ACTIVITY

4.4.1: Section review questions.



The ____ determines the decision of which kernel thread to schedule onto which CPU.



- ☐ Process-contention scope
- ☐ System-contention scope
- ☐ Dispatcher

Section glossary

process-contention scope (PCS): On systems implementing the many-to-one and many-to-many threading models, the thread library schedules user-level threads to run on an available LWP (and thus threads contend with others within the same process contend for CPU time).

system-contention scope (SCS): A thread scheduling method in which kernel-level threads are scheduled onto a CPU, regardless of which process they are associated with (and thus contending with all other threads on the system for CPU time).

4.5 Multi-processor scheduling

Our discussion thus far has focused on the problems of scheduling the CPU in a system with a single processing core. If multiple CPUs are available, **load sharing**, where multiple threads may run in parallel, becomes possible, however scheduling issues become correspondingly more complex. Many possibilities have been tried; and as we saw with CPU scheduling with a single-core CPU, there is no one best solution.

Traditionally, the term **multiprocessor** referred to systems that provided multiple physical processors, where each processor contained one single-core CPU. However, the definition of multiprocessor has evolved significantly, and on modern computing systems, **multiprocessor** now applies to the following system architectures:

- Multicore CPUs
- Multithreaded cores
- NUMA systems
- Heterogeneous multiprocessing

Here, we discuss several concerns in multiprocessor scheduling in the context of these different architectures. In the first three examples we concentrate on systems in which the processors are identical—homogeneous—in terms of their functionality. We can then use any available CPU to run any process in the queue. In the last example we explore a system where the processors are not identical in their capabilities.

Approaches to multiple-processor scheduling

One approach to CPU scheduling in a multiprocessor system has all scheduling decisions, I/O processing, and other system activities handled by a single processor—the main server. The other processors execute only user code. This **asymmetric multiprocessing** is simple because only one core accesses the system data structures, reducing the need for data sharing. The downfall of this approach is the main server becomes a potential bottleneck where overall system performance may be reduced.

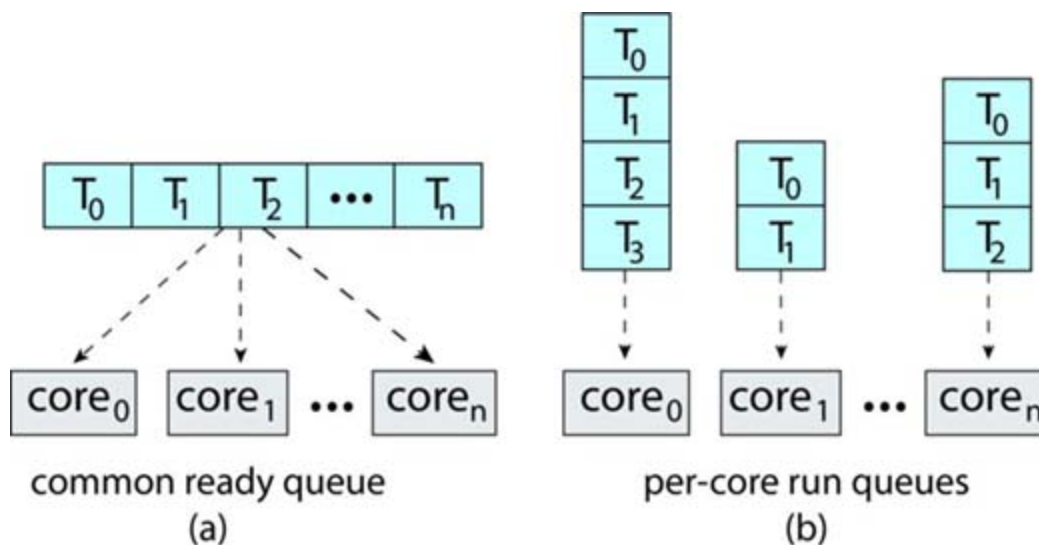
The standard approach for supporting multiprocessors is **symmetric multiprocessing (SMP)**, where each processor is self-scheduling. Scheduling proceeds by having the scheduler for each processor examine the ready queue and select a thread to run. Note that this provides two possible strategies for organizing the threads eligible to be scheduled:

1. All threads may be in a common ready queue.

2. Each processor may have its own private queue of threads.

These two strategies are contrasted in Figure 4.5.1. If we select the first option, we have a possible race condition on the shared ready queue and therefore must ensure that two separate processors do not choose to schedule the same thread and that threads are not lost from the queue. As discussed in the chapter Synchronization Tools, we could use some form of locking to protect the common ready queue from this race condition. Locking would be highly contended, however, as all accesses to the queue would require lock ownership, and accessing the shared queue would likely be a performance bottleneck. The second option permits each processor to schedule threads from its private run queue and therefore does not suffer from the possible performance problems associated with a shared run queue. Thus, it is the most common approach on systems supporting SMP. Additionally, as described in Section Processor affinity, having private, per-processor run queues in fact may lead to more efficient use of cache memory. There are issues with per-processor run queues—most notably, workloads of varying sizes. However, as we shall see, balancing algorithms can be used to equalize workloads among all processors.

Figure 4.5.1: Organization of ready queues.



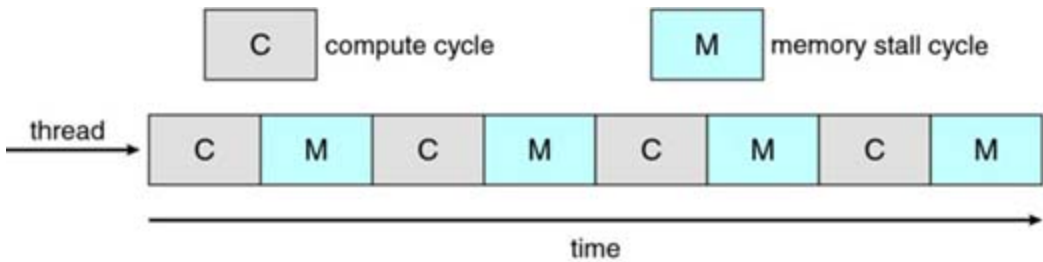
Virtually all modern operating systems support SMP, including Windows, Linux, and macOS as well as mobile systems including Android and iOS. In the remainder of this section, we discuss issues concerning SMP systems when designing CPU scheduling algorithms.

Multicore processors

Traditionally, SMP systems have allowed several processes to run in parallel by providing multiple physical processors. However, most contemporary computer hardware now places multiple computing cores on the same physical chip, resulting in a **multicore processor**. Each core maintains its architectural state and thus appears to the operating system to be a separate logical CPU. SMP systems that use multicore processors are faster and consume less power than systems in which each CPU has its own physical chip.

Multicore processors may complicate scheduling issues. Let's consider how this can happen. Researchers have discovered that when a processor accesses memory, it spends a significant amount of time waiting for the data to become available. This situation, known as a **memory stall**, occurs primarily because modern processors operate at much faster speeds than memory. However, a memory stall can also occur because of a cache miss (accessing data that are not in cache memory). Figure 4.5.2 illustrates a memory stall. In this scenario, the processor can spend up to 50 percent of its time waiting for data to become available from memory.

Figure 4.5.2: Memory stall.



To remedy this situation, many recent hardware designs have implemented multithreaded processing cores in which two (or more) **hardware threads** are assigned to each core. That way, if one hardware thread stalls while waiting for memory, the core can switch to another thread. Figure 4.5.3 illustrates a dual-threaded processing core on which the execution of thread 0 and the execution of thread 1 are interleaved. From an operating system perspective, each hardware thread maintains its architectural state, such as instruction pointer and register set, and thus appears as a logical CPU that is available to run a software thread. This technique—known as **chip multithreading** (CMT)—is illustrated in Figure 4.5.4. Here, the processor contains four computing cores, with each core containing two hardware threads. From the perspective of the operating system, there are eight logical CPUs.

Figure 4.5.3: Multithreaded multicore system.

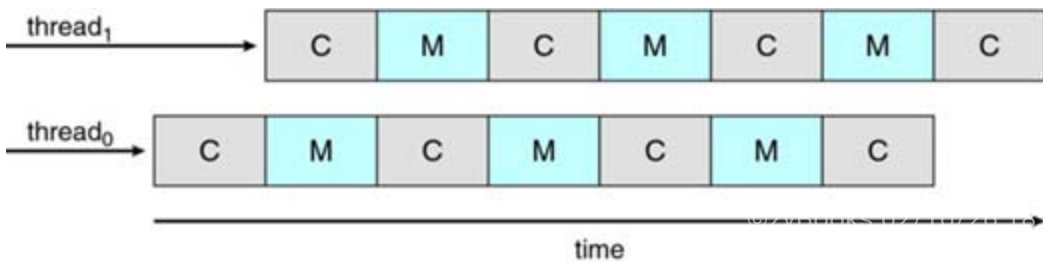
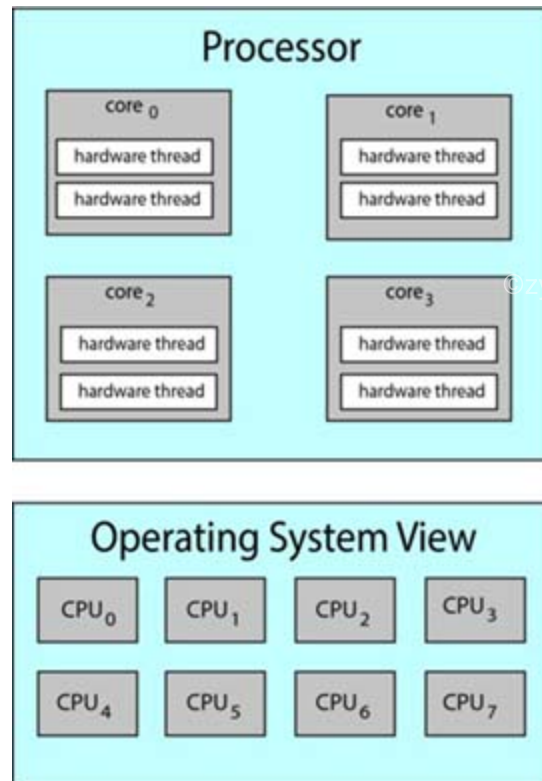


Figure 4.5.4: Chip multithreading.

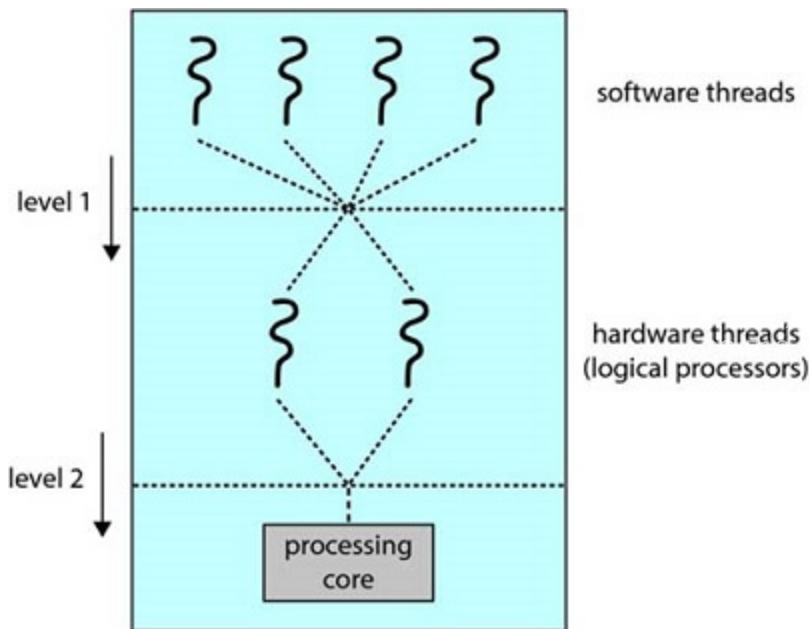


Intel processors use the term **hyper-threading** (also known as **simultaneous multithreading** or SMT) to describe assigning multiple hardware threads to a single processing core. Contemporary Intel processors—such as the i7—support two threads per core, while the Oracle Sparc M7 processor supports eight threads per core, with eight cores per processor, thus providing the operating system with 64 logical CPUs.

In general, there are two ways to multithread a processing core: **coarse-grained** and **fine-grained** multithreading. With coarse-grained multithreading, a thread executes on a core until a long-latency event such as a memory stall occurs. Because of the delay caused by the long-latency event, the core must switch to another thread to begin execution. However, the cost of switching between threads is high, since the instruction pipeline must be flushed before the other thread can begin execution on the processor core. Once this new thread begins execution, it begins filling the pipeline with its instructions. Fine-grained (or interleaved) multithreading switches between threads at a much finer level of granularity—typically at the boundary of an instruction cycle. However, the architectural design of fine-grained systems includes logic for thread switching. As a result, the cost of switching between threads is small.

It is important to note that the resources of the physical core (such as caches and pipelines) must be shared among its hardware threads, and therefore a processing core can only execute one hardware thread at a time. Consequently, a multithreaded, multicore processor actually requires two different levels of scheduling, as shown in Figure 4.5.5, which illustrates a dual-threaded processing core.

Figure 4.5.5: Two levels of scheduling.



On one level are the scheduling decisions that must be made by the operating system as it chooses which software thread to run on each hardware thread (logical CPU). For all practical purposes, such decisions have been the primary focus of this chapter. Therefore, for this level of scheduling, the operating system may choose any scheduling algorithm, including those described in Section [4.3](#).

A second level of scheduling specifies how each core decides which hardware thread to run. There are several strategies to adopt in this situation. One approach is to use a simple round-robin algorithm to schedule a hardware thread to the processing core. This is the approach adopted by the UltraSPARC T3. Another approach is used by the Intel Itanium, a dual-core processor with two hardware-managed threads per core. Assigned to each hardware thread is a dynamic **urgency** value ranging from 0 to 7, with 0 representing the lowest urgency and 7 the highest. The Itanium identifies five different events that may trigger a thread switch. When one of these events occurs, the thread-switching logic compares the urgency of the two threads and selects the thread with the highest urgency value to execute on the processor core.

Note that the two different levels of scheduling shown in Figure [4.5.5](#) are not necessarily mutually exclusive. In fact, if the operating system scheduler (the first level) is made aware of the sharing of processor resources, it can make more effective scheduling decisions. As an example, assume that a CPU has two processing cores, and each core has two hardware threads. If two software threads are running on this system, they can be running either on the same core or on separate cores. If they are both scheduled to run on the same core, they have to share processor resources and thus are likely to proceed more slowly than if they were scheduled on separate cores. If the operating system is aware of the level of processor resource sharing, it can schedule software threads onto logical processors that do not share resources.



How many threads may run in parallel on a single processing core with two hardware threads?

- ☐ 1
- ☐ 2
- ☐ 4

Load balancing

On SMP systems, it is important to keep the workload balanced among all processors to fully utilize the benefits of having more than one processor. Otherwise, one or more processors may sit idle while other processors have high workloads, along with ready queues of threads awaiting the CPU. **Load balancing** attempts to keep the workload evenly distributed across all processors in an SMP system. It is important to note that load balancing is typically necessary only on systems where each processor has its own private ready queue of eligible threads to execute. On systems with a common run queue, load balancing is unnecessary, because once a processor becomes idle, it immediately extracts a runnable thread from the common ready queue.

There are two general approaches to load balancing: push migration and pull migration. With **push migration**, a specific task periodically checks the load on each processor and—if it finds an imbalance—evenly distributes the load by moving (or pushing) threads from overloaded to idle or less-busy processors. **Pull migration** occurs when an idle processor pulls a waiting task from a busy processor. Push and pull migration need not be mutually exclusive and are, in fact, often implemented in parallel on load-balancing systems. For example, the Linux CFS scheduler (described in Section Example: Linux scheduling) and the ULE scheduler available for FreeBSD systems implement both techniques.

The concept of a "balanced load" may have different meanings. One view of a balanced load may require simply that all queues have approximately the same number of threads. Alternatively, balance may require an equal distribution of thread priorities across all queues. In addition, in certain situations, neither of these strategies may be sufficient. Indeed, they may work against the goals of the scheduling algorithm. (We leave further consideration of this as an exercise.)

Processor affinity

Consider what happens to cache memory when a thread has been running on a specific processor. The data most recently accessed by the thread populate the cache for the processor. As a result, successive memory accesses by the thread are often satisfied in cache memory (known as a "warm cache"). Now consider what happens if the thread migrates to another processor—say, due to load balancing. The contents of cache memory must be invalidated for the first processor, and the cache for the second processor must be repopulated. Because of the high cost of invalidating and repopulating caches, most operating systems with SMP support try to avoid migrating a thread from one processor to another and instead attempt to keep a thread running on the same processor and take advantage of a warm cache.

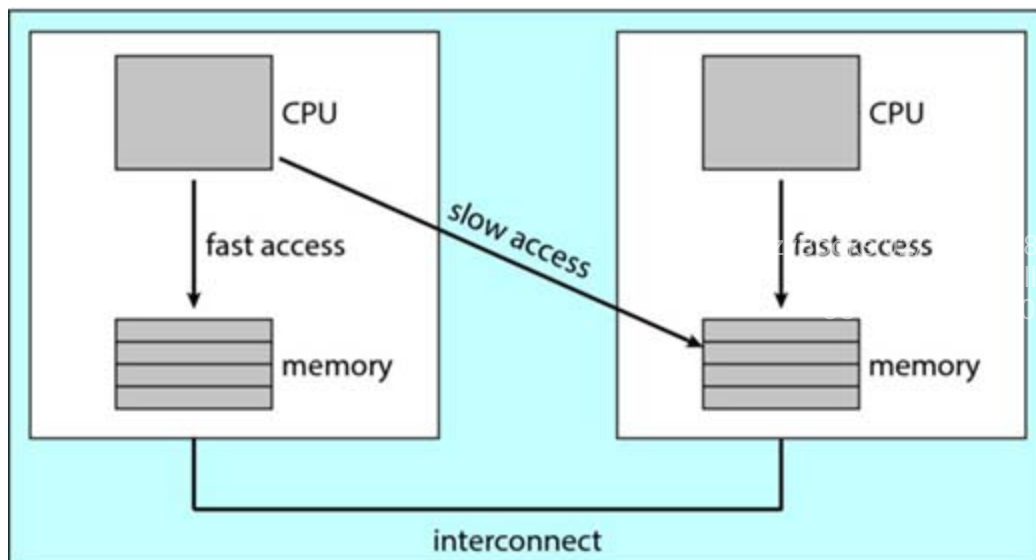
This is known as **processor affinity**—that is, a process has an affinity for the processor on which it is currently running.

The two strategies described in Section Approaches to multiple-processor scheduling for organizing the queue of threads available for scheduling have implications for processor affinity. If we adopt the approach of a common ready queue, a thread may be selected for execution by any processor. Thus, if a thread is scheduled on a new processor, that processor's cache must be repopulated. With private, per-processor ready queues, a thread is always scheduled on the same processor and can therefore benefit from the contents of a warm cache. Essentially, per-processor ready queues provide processor affinity for free!

Processor affinity takes several forms. When an operating system has a policy of attempting to keep a process running on the same processor—but not guaranteeing that it will do so—we have a situation known as **soft affinity**. Here, the operating system will attempt to keep a process on a single processor, but it is possible for a process to migrate between processors during load balancing. In contrast, some systems provide system calls that support **hard affinity**, thereby allowing a process to specify a subset of processors on which it can run. Many systems provide both soft and hard affinity. For example, Linux implements soft affinity, but it also provides the `sched_setaffinity()` system call, which supports hard affinity by allowing a thread to specify the set of CPUs on which it is eligible to run.

The main-memory architecture of a system can affect processor affinity issues as well. Figure 4.5.6 illustrates an architecture featuring non-uniform memory access (NUMA) where there are two physical processor chips each with their own CPU and local memory. Although a system interconnect allows all CPUs in a NUMA system to share one physical address space, a CPU has faster access to its local memory than to memory local to another CPU. If the operating system's CPU scheduler and memory-placement algorithms are **NUMA-aware** and work together, then a thread that has been scheduled onto a particular CPU can be allocated memory closest to where the CPU resides, thus providing the thread the fastest possible memory access.

Figure 4.5.6: NUMA and CPU scheduling.



Interestingly, load balancing often counteracts the benefits of processor affinity. That is, the benefit of keeping a thread running on the same processor is that the thread can take advantage of its data being in that processor's cache memory. Balancing loads by moving a thread from one processor to another removes this benefit. Similarly, migrating a thread between processors may incur a penalty on NUMA systems, where a thread may be moved to a processor that requires longer memory access times. In other words, there is a natural tension between load balancing and minimizing memory access times. Thus, scheduling algorithms for modern multicore NUMA systems have become quite complex. In Section Example: Linux scheduling, we examine the Linux CFS scheduling algorithm and explore how it balances these competing goals.

**PARTICIPATION
ACTIVITY**

4.5.2: Mid-section review question.



___ means that a process has an affinity for the processor on which it is currently running.



Check

[Show answer](#)

Heterogeneous multiprocessing

In the examples we have discussed so far, all processors are identical in terms of their capabilities, thus allowing any thread to run on any processing core. The only difference being that memory access times may vary based upon load balancing and processor affinity policies, as well as on NUMA systems.

Although mobile systems now include multicore architectures, some systems are now designed using cores that run the same instruction set, yet vary in terms of their clock speed and power management, including the ability to adjust the power consumption of a core to the point of idling the core. Such systems are known as **heterogeneous multiprocessing** (HMP). Note this is not a form of asymmetric multiprocessing as described in Section Approaches to multiple-processor scheduling as both system and user tasks can run on any core. Rather, the intention behind HMP is to better manage power consumption by assigning tasks to certain cores based upon the specific demands of the task.

For ARM processors that support it, this type of architecture is known as **big.LITTLE** where higher-performance **big** cores are combined with energy efficient **LITTLE** cores. **Big** cores consume greater energy and therefore should only be used for short periods of time. Likewise, **little** cores use less energy and can therefore be used for longer periods.

There are several advantages to this approach. By combining a number of slower cores with faster ones, a CPU scheduler can assign tasks that do not require high performance, but may need to run for longer periods, (such as background tasks) to little cores, thereby helping to preserve a battery charge. Similarly, interactive applications which require more processing power, but may run for shorter durations, can be

assigned to big cores. Additionally, if the mobile device is in a power-saving mode, energy-intensive big cores can be disabled and the system can rely solely on energy-efficient little cores. Windows 10 supports HMP scheduling by allowing a thread to select a scheduling policy that best supports its power management demands.

**PARTICIPATION
ACTIVITY**

4.5.3: Section review questions.



- 1) With ____ a thread executes on a processor until a long-latency event (i.e. a memory stall) occurs, at which point the processor switches to another thread.
- ☐ coarse-grained multithreading
 - ☐ fine-grained multithreading
 - ☐ symmetric multiprocessing (SMP)
- 2) The two general approaches to load balancing are ____ and ____.
- ☐ coarse grained, fine grained
 - ☐ soft real-time, hard real-time
 - ☐ push migration, pull migration



Section glossary

load sharing: The ability of a system with multiple CPU cores to schedule threads on those cores.

asymmetric multiprocessing: A simple multiprocessor scheduling algorithm in which only one processor accesses the system data structures and others run user threads, reducing the need for data sharing.

symmetric multiprocessing: A mutliprocessor scheduling method, where each processor is self-scheduling and may run kernel threads or user level threads (contending for access to kernel data structures and requiring locking).

memory stall: When a thread is on CPU and accesses memory contents that is not in the CPU's cache, the thread execution stalls while the contents of that memory is fetched.

hardware threads: A given CPU core can run a single thread (one hardware thread per core) or more than one per core (multiple hardware threads per core) to optimize core use, for example to avoid memory stalls by switching hardware threads if the current thread causes a stall.

chip multithreading (CMT): A CPU with multiple cores, where each core supports multiple hardware threads supports chip multithreading.

hyper-threading: See chip multithreading.

simultaneous multithreading (SMT): See chip multithreading.

Load balancing: Load balancing attempts to keep the workload evenly distributed across all processors in an SMP system.

push migration: With push migration, a specific task periodically checks the load on each processor and—if it finds an imbalance—evenly distributes the load by moving (or pushing) threads from overloaded to idle or less-busy processors.

pull migration: Pull migration occurs when an idle processor pulls a waiting thread from a busy processor.

processor affinity: A kernel scheduling method in which a process has an affinity for the processor in which it is currently running (to keep the cache warm for example).

soft affinity: When an operating system has a policy of attempting to keep a process running on the same processor—but not guaranteeing that it will do so.

hard affinity: When an operating system supports or allows a process's threads to run on the same processor at all times (as opposed to being moved to various processors as the thread is scheduled onto CPU).

heterogeneous multiprocessing (HMP): A feature of some mobile computing CPUs in which cores vary in their clock speed and power management.

big.LITTLE: ARM processor implementation of HMP in which high performance big cores are combined with energy efficient LITTLE cores.

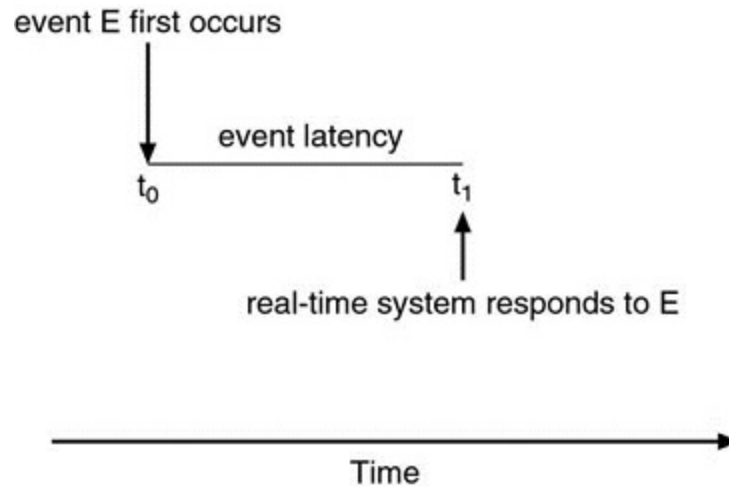
4.6 Real-time CPU scheduling

CPU scheduling for real-time operating systems involves special issues. In general, we can distinguish between soft real-time systems and hard real-time systems. **Soft real-time systems** provide no guarantee as to when a critical real-time process will be scheduled. They guarantee only that the process will be given preference over noncritical processes. **Hard real-time systems** have stricter requirements. A task must be serviced by its deadline; service after the deadline has expired is the same as no service at all. In this section, we explore several issues related to process scheduling in both soft and hard real-time operating systems.

Minimizing latency

Consider the event-driven nature of a real-time system. The system is typically waiting for an event in real time to occur. Events may arise either in software—as when a timer expires—or in hardware—as when a remote-controlled vehicle detects that it is approaching an obstruction. When an event occurs, the system must respond to and service it as quickly as possible. We refer to **event latency** as the amount of time that elapses from when an event occurs to when it is serviced (Figure [4.6.1](#)).

Figure 4.6.1: Event latency.



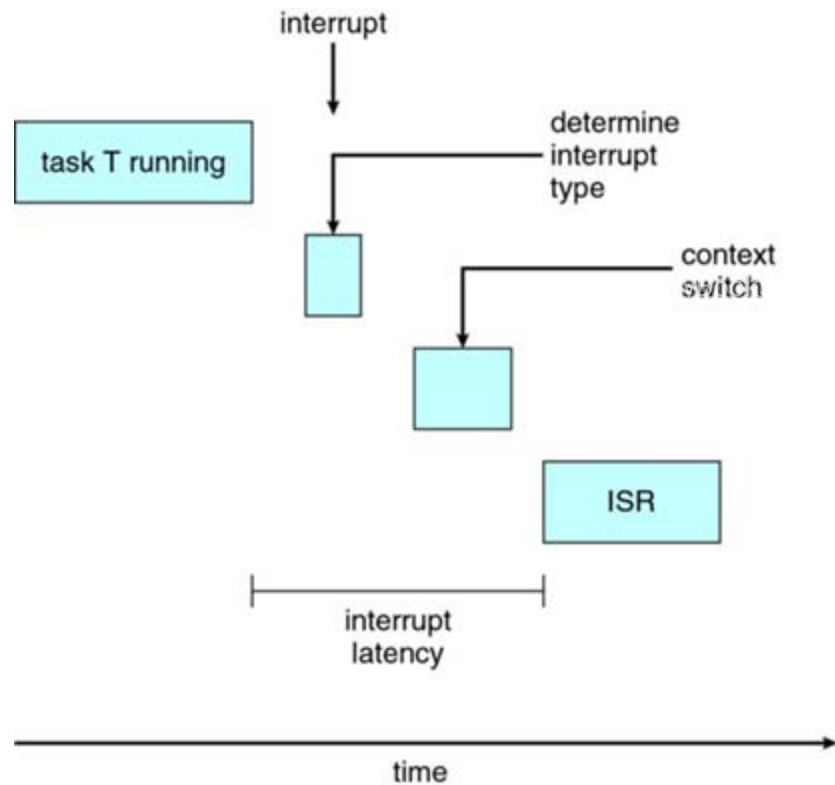
Usually, different events have different latency requirements. For example, the latency requirement for an antilock brake system might be 3 to 5 milliseconds. That is, from the time a wheel first detects that it is sliding, the system controlling the antilock brakes has 3 to 5 milliseconds to respond to and control the situation. Any response that takes longer might result in the automobile's veering out of control. In contrast, an embedded system controlling radar in an airliner might tolerate a latency period of several seconds.

Two types of latencies affect the performance of real-time systems:

1. Interrupt latency
2. Dispatch latency

Interrupt latency refers to the period of time from the arrival of an interrupt at the CPU to the start of the routine that services the interrupt. When an interrupt occurs, the operating system must first complete the instruction it is executing and determine the type of interrupt that occurred. It must then save the state of the current process before servicing the interrupt using the specific interrupt service routine (ISR). The total time required to perform these tasks is the interrupt latency (Figure [4.6.2](#)).

Figure 4.6.2: Interrupt latency.



Obviously, it is crucial for real-time operating systems to minimize interrupt latency to ensure that real-time tasks receive immediate attention. Indeed, for hard real-time systems, interrupt latency must not simply be minimized, it must be bounded to meet the strict requirements of these systems.

One important factor contributing to interrupt latency is the amount of time interrupts may be disabled while kernel data structures are being updated. Real-time operating systems require that interrupts be disabled for only very short periods of time.

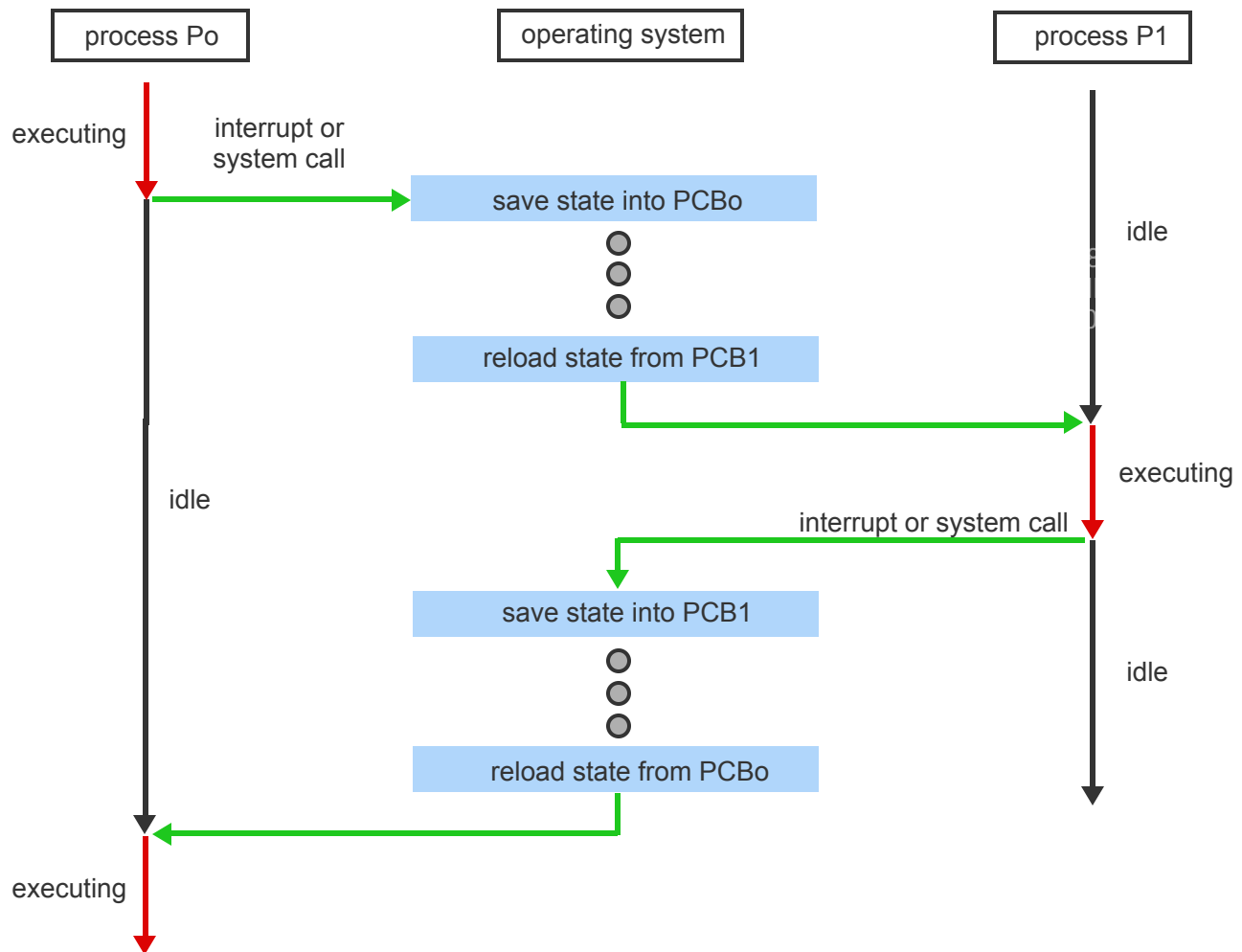
The amount of time required for the scheduling dispatcher to stop one process and start another is known as **dispatch latency**. Providing real-time tasks with immediate access to the CPU mandates that real-time operating systems minimize this latency as well. The most effective technique for keeping dispatch latency low is to provide preemptive kernels. For hard real-time systems, dispatch latency is typically measured in several microseconds.

In the animation below, we diagram the makeup of dispatch latency. The **conflict phase** of dispatch latency has two components:

1. Preemption of any process running in the kernel
2. Release by low-priority processes of resources needed by a high-priority process

Following the conflict phase, the dispatch phase schedules the high-priority process onto an available CPU.





Animation content:

Step 1: The system has two processes, P0 and P1; P1 is idle, P0 is executing and executes a system call or the system receives an interrupt the operating system saves the state of P0 in its PCB. Step 2: P0 is now off core, so idle, and the operating system restores the state of P1 from its PCB puts P1 on the core and it continues execution. Step 3: P1 execution is interrupted, the operating system saves its state to its PCB and restores the next process's state (P0 in this case) to prepare it to continue execution. Step 4: The operating system puts P0 onto the core, continuing its execution while P1 is now idle, in ready state, waiting its turn.

Animation captions:

1. The system has two processes, P0 and P1; P1 is idle, P0 is executing and executes a system call or the system receives an interrupt the operating system saves the state of P0 in its PCB.
2. P0 is now off core, so idle, and the operating system restores the state of P1 from its PCB puts P1 on the core and it continues execution.

3. P1 execution is interrupted, the operating system saves its state to its PCB and restores the next process's state (P0 in this case) to prepare it to continue execution.
4. The operating system puts P0 onto the core, continuing its execution while P1 is now idle, in ready state, waiting its turn.

Priority-based scheduling

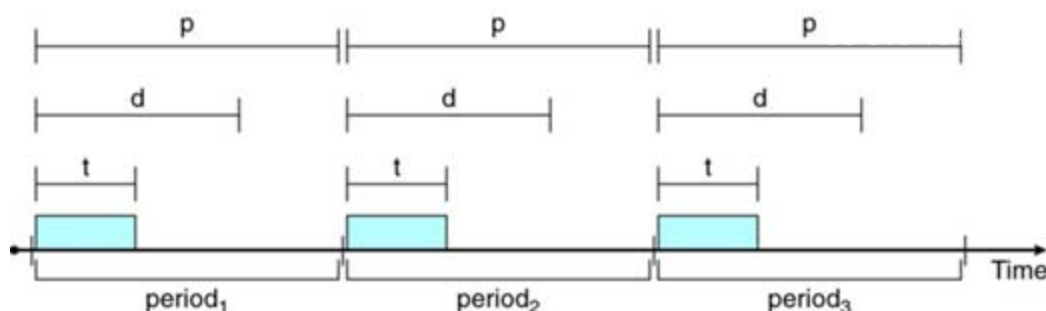
The most important feature of a real-time operating system is to respond immediately to a real-time process as soon as that process requires the CPU. As a result, the scheduler for a real-time operating system must support a priority-based algorithm with preemption. Recall that priority-based scheduling algorithms assign each process a priority based on its importance; more important tasks are assigned higher priorities than those deemed less important. If the scheduler also supports preemption, a process currently running on the CPU will be preempted if a higher-priority process becomes available to run.

Preemptive, priority-based scheduling algorithms are discussed in detail in Section Priority scheduling, and Section [Missing Reference: Operating-system examples](#) presents examples of the soft real-time scheduling features of the Linux, Windows, and Solaris operating systems. Each of these systems assigns real-time processes the highest scheduling priority. For example, Windows has 32 different priority levels. The highest levels—priority values 16 to 31—are reserved for real-time processes. Solaris and Linux have similar prioritization schemes.

Note that providing a preemptive, priority-based scheduler only guarantees soft real-time functionality. Hard real-time systems must further guarantee that real-time tasks will be serviced in accord with their deadline requirements, and making such guarantees requires additional scheduling features. In the remainder of this section, we cover scheduling algorithms appropriate for hard real-time systems.

Before we proceed with the details of the individual schedulers, however, we must define certain characteristics of the processes that are to be scheduled. First, the processes are considered **periodic**. That is, they require the CPU at constant intervals (periods). Once a periodic process has acquired the CPU, it has a fixed processing time t , a deadline d by which it must be serviced by the CPU, and a period p . The relationship of the processing time, the deadline, and the period can be expressed as $0 \leq t \leq d \leq p$. The **rate** of a periodic task is $1/p$. Figure 4.6.3 illustrates the execution of a periodic process over time. Schedulers can take advantage of these characteristics and assign priorities according to a process's deadline or rate requirements.

Figure 4.6.3: Periodic task.



What is unusual about this form of scheduling is that a process may have to announce its deadline requirements to the scheduler. Then, using a technique known as an **admission-control** algorithm, the scheduler does one of two things. It either admits the process, guaranteeing that the process will complete on time, or rejects the request as impossible if it cannot guarantee that the task will be serviced by its deadline.

Rate-monotonic scheduling

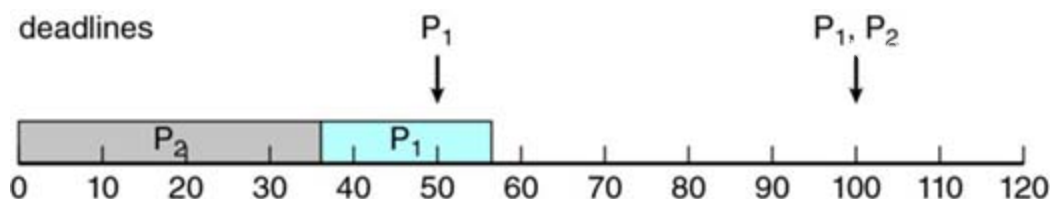
The **rate-monotonic** scheduling algorithm schedules periodic tasks using a static priority policy with preemption. If a lower-priority process is running and a higher-priority process becomes available to run, it will preempt the lower-priority process. Upon entering the system, each periodic task is assigned a priority inversely based on its period. The shorter the period, the higher the priority; the longer the period, the lower the priority. The rationale behind this policy is to assign a higher priority to tasks that require the CPU more often. Furthermore, rate-monotonic scheduling assumes that the processing time of a periodic process is the same for each CPU burst. That is, every time a process acquires the CPU, the duration of its CPU burst is the same.

Let's consider an example. We have two processes, P_1 and P_2 . The periods for P_1 and P_2 are 50 and 100, respectively—that is, $P_1 = 50$ and $P_2 = 100$. The processing times are $t_1 = 20$ for P_1 and $t_2 = 35$ for P_2 . The deadline for each process requires that it complete its CPU burst by the start of its next period.

We must first ask ourselves whether it is possible to schedule these tasks so that each meets its deadlines. If we measure the CPU utilization of a process P_i as the ratio of its burst to its period— t_i/p_i —the CPU utilization of P_1 is $20 / 50 = 0.40$ and that of P_2 is $35 / 100 = 0.35$, for a total CPU utilization of 75 percent. Therefore, it seems we can schedule these tasks in such a way that both meet their deadlines and still leave the CPU with available cycles.

Suppose we assign P_2 a higher priority than P_1 . The execution of P_1 and P_2 in this situation is shown in Figure 4.6.4. As we can see, P_2 starts execution first and completes at time 35. At this point, P_1 starts; it completes its CPU burst at time 55. However, the first deadline for P_1 was at time 50, so the scheduler has caused P_1 to miss its deadline.

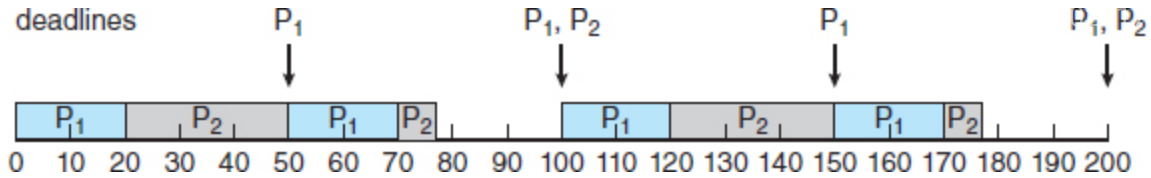
Figure 4.6.4: Scheduling of tasks when P_2 has a higher priority than P_1 .



Now suppose we use rate-monotonic scheduling, in which we assign P_1 a higher priority than P_2 because the period of P_1 is shorter than that of P_2 . The execution of these processes in this situation is shown in Figure 4.6.5. P_1 starts first and completes its CPU burst at time 20, thereby meeting its first deadline. P_2

starts running at this point and runs until time 50. At this time, it is preempted by P_1 , although it still has 5 milliseconds remaining in its CPU burst. P_1 completes its CPU burst at time 70, at which point the scheduler resumes P_2 . P_2 completes its CPU burst at time 75, also meeting its first deadline. The system is idle until time 100, when P_1 is scheduled again.

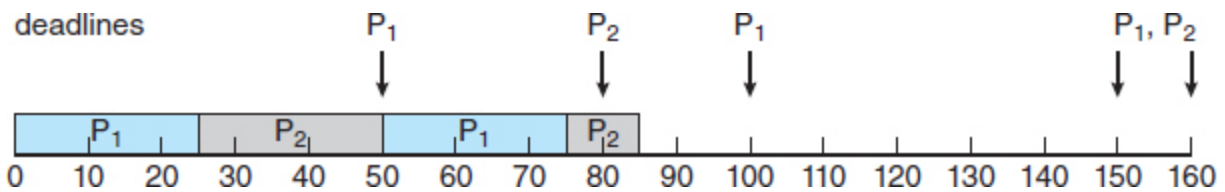
Figure 4.6.5: Rate-monotonic scheduling.



Rate-monotonic scheduling is considered optimal in that if a set of processes cannot be scheduled by this algorithm, it cannot be scheduled by any other algorithm that assigns static priorities. Let's next examine a set of processes that cannot be scheduled using the rate-monotonic algorithm.

Assume that process P_1 has a period of $p_1 = 50$ and a CPU burst of $t_1 = 25$. For P_2 , the corresponding values are $p_2 = 80$ and $t_2 = 35$. Rate-monotonic scheduling would assign process P_1 a higher priority, as it has the shorter period. The total CPU utilization of the two processes is $(25 / 50) + (35 / 80) = 0.94$, and it therefore seems logical that the two processes could be scheduled and still leave the CPU with 6 percent available time. Figure 4.6.6 shows the scheduling of processes P_1 and P_2 . Initially, P_1 runs until it completes its CPU burst at time 25. Process P_2 then begins running and runs until time 50, when it is preempted by P_1 . At this point, P_2 still has 10 milliseconds remaining in its CPU burst. Process P_1 runs until time 75; consequently, P_2 finishes its burst at time 85, after the deadline for completion of its CPU burst at time 80.

Figure 4.6.6: Missing deadlines with rate-monotonic scheduling.



Despite being optimal, then, rate-monotonic scheduling has a limitation: CPU utilization is bounded, and it is not always possible to maximize CPU resources fully. The worst-case CPU utilization for scheduling N processes is

$$N \left(2^{1/N} - 1 \right).$$

With one process in the system, CPU utilization is 100 percent, but it falls to approximately 69 percent as the number of processes approaches infinity. With two processes, CPU utilization is bounded at about 83

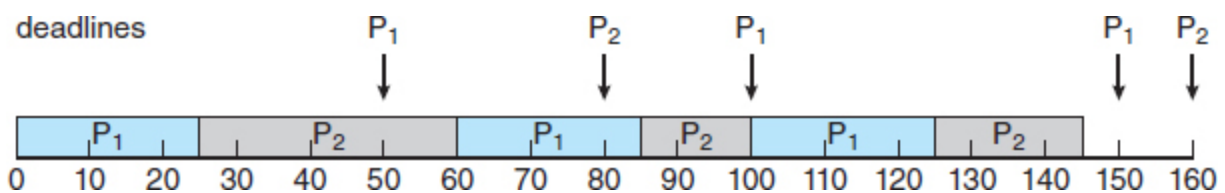
percent. Combined CPU utilization for the two processes scheduled in Figure 4.6.4 and Figure 4.6.5 is 75 percent; therefore, the rate-monotonic scheduling algorithm is guaranteed to schedule them so that they can meet their deadlines. For the two processes scheduled in Figure 4.6.6, combined CPU utilization is approximately 94 percent; therefore, rate-monotonic scheduling cannot guarantee that they can be scheduled so that they meet their deadlines.

Earliest-deadline-first scheduling

Earliest-deadline-first (EDF) scheduling assigns priorities dynamically according to deadline. The earlier the deadline, the higher the priority; the later the deadline, the lower the priority. Under the EDF policy, when a process becomes runnable, it must announce its deadline requirements to the system. Priorities may have to be adjusted to reflect the deadline of the newly runnable process. Note how this differs from rate-monotonic scheduling, where priorities are fixed.

To illustrate EDF scheduling, we again schedule the processes shown in Figure 4.6.6, which failed to meet deadline requirements under rate-monotonic scheduling. Recall that P_1 has values of $p_1 = 50$ and $t_1 = 25$ and that P_2 has values of $p_2 = 80$ and $t_2 = 35$. The EDF scheduling of these processes is shown in Figure 4.6.7. Process P_1 has the earliest deadline, so its initial priority is higher than that of process P_2 . Process P_2 begins running at the end of the CPU burst for P_1 . However, whereas rate-monotonic scheduling allows P_1 to preempt P_2 at the beginning of its next period at time 50, EDF scheduling allows process P_2 to continue running. P_2 now has a higher priority than P_1 because its next deadline (at time 80) is earlier than that of P_1 (at time 100). Thus, both P_1 and P_2 meet their first deadlines. Process P_1 again begins running at time 60 and completes its second CPU burst at time 85, also meeting its second deadline at time 100. P_2 begins running at this point, only to be preempted by P_1 at the start of its next period at time 100. P_2 is preempted because P_1 has an earlier deadline (time 150) than P_2 (time 160). At time 125, P_1 completes its CPU burst and P_2 resumes execution, finishing at time 145 and meeting its deadline as well. The system is idle until time 150, when P_1 is scheduled to run once again.

Figure 4.6.7: Earliest-deadline-first scheduling.



Unlike the rate-monotonic algorithm, EDF scheduling does not require that processes be periodic, nor must a process require a constant amount of CPU time per burst. The only requirement is that a process announce its deadline to the scheduler when it becomes runnable. The appeal of EDF scheduling is that it is theoretically optimal—theoretically, it can schedule processes so that each process can meet its deadline requirements and CPU utilization will be 100 percent. In practice, however, it is impossible to achieve this level of CPU utilization due to the cost of context switching between processes and interrupt handling.

Proportional share scheduling

Proportional share schedulers operate by allocating T shares among all applications. An application can receive N shares of time, thus ensuring that the application will have N / T of the total processor time. As an example, assume that a total of $T = 100$ shares is to be divided among three processes, A , B , and C . A is assigned 50 shares, B is assigned 15 shares, and C is assigned 20 shares. This scheme ensures that A will have 50 percent of total processor time, B will have 15 percent, and C will have 20 percent.

Proportional share schedulers must work in conjunction with an admission-control policy to guarantee that an application receives its allocated shares of time. An admission-control policy will admit a client requesting a particular number of shares only if sufficient shares are available. In our current example, we have allocated $50 + 15 + 20 = 85$ shares of the total of 100 shares. If a new process D requested 30 shares, the admission controller would deny D entry into the system.

POSIX real-time scheduling

The POSIX standard also provides extensions for real-time computing—POSIX.1b. Here, we cover some of the POSIX API related to scheduling real-time threads. POSIX defines two scheduling classes for real-time threads:

- `SCHED_FIFO`
- `SCHED_RR`

`SCHED_FIFO` schedules threads according to a first-come, first-served policy using a FIFO queue as outlined in Section First-come, first-served scheduling. However, there is no time slicing among threads of equal priority. Therefore, the highest-priority real-time thread at the front of the FIFO queue will be granted the CPU until it terminates or blocks. `SCHED_RR` uses a round-robin policy. It is similar to `SCHED_FIFO` except that it provides time slicing among threads of equal priority. POSIX provides an additional scheduling class—`SCHED_OTHER`—but its implementation is undefined and system specific; it may behave differently on different systems.

The POSIX API specifies the following two functions for getting and setting the scheduling policy:

- `pthread_attr_getschedpolicy(pthread_attr_t *attr, int *policy)`
- `pthread_attr_setschedpolicy(pthread_attr_t *attr, int policy)`

The first parameter to both functions is a pointer to the set of attributes for the thread. The second parameter is either (1) a pointer to an integer that is set to the current scheduling policy (for `pthread_attr_getsched_policy()`) or (2) an integer value (`SCHED_FIFO`, `SCHED_RR`, or `SCHED_OTHER`) for the `pthread_attr_setsched_policy()` function. Both functions return nonzero values if an error occurs.

In Figure [4.6.8](#), we illustrate a POSIX Pthread program using this API. This program first determines the current scheduling policy and then sets the scheduling algorithm to `SCHED_FIFO`.

Figure 4.6.8: POSIX real-time scheduling API.

```
#include <pthread.h>
#include <stdio.h>
#define NUM_THREADS 5

int main(int argc, char *argv[])
{
    int i, policy;
    pthread_t tid[NUM_THREADS];
    pthread_attr_t attr;

    /* get the default attributes */
    pthread_attr_init(&attr);

    /* get the current scheduling policy */
    if (pthread_attr_getschedpolicy(&attr, &policy) != 0)
        fprintf(stderr, "Unable to get policy.\n");
    else {
        if (policy == SCHED_OTHER)
            printf("SCHED_OTHER\n");
        else if (policy == SCHED_RR)
            printf("SCHED_RR\n");
        else if (policy == SCHED_FIFO)
            printf("SCHED_FIFO\n");
    }

    /* set the scheduling policy - FIFO, RR, or OTHER */
    if (pthread_attr_setschedpolicy(&attr, SCHED_FIFO) != 0)
        fprintf(stderr, "Unable to set policy.\n");

    /* create the threads */
    for (i = 0; i < NUM_THREADS; i++)
        pthread_create(&tid[i], &attr, runner, NULL);

    /* now join on each thread */
    for (i = 0; i < NUM_THREADS; i++)
        pthread_join(tid[i], NULL);
}

/* Each thread will begin control in this function */
void *runner(void *param)
{
    /* do some work ... */
    pthread_exit(0);
}
```

©zyBooks 02/10/26 18:26 2150642
Adrian Tull
CS-510-10435.202617-1

©zyBooks 02/10/26 18:26 2150642



1) The rate of a periodic task in a hard real-time system is ____, where ***p*** is a period and ***t*** is the processing time.

☐ **$1/p$**

☐ **p/t**

☐ **$1/t$**

2) Which of the following is true of the rate-monotonic scheduling algorithm?

☐ It uses a dynamic priority policy.

☐ CPU utilization is bounded when using this algorithm.

☐ It is non-preemptive.

3) Which of the following is true of earliest-deadline-first (EDF) scheduling algorithm?

☐ When a process becomes runnable, the process must announce its deadline requirements to the system.

☐ Priorities are assigned as follows: the earlier the deadline, the lower the priority; the later the deadline, the higher the priority.

☐ Priorities are fixed; that is, they cannot be adjusted when a new process starts running.

Section glossary

soft real-time systems: Soft real-time systems provide no guarantee as to when a critical real-time thread will be scheduled - they guarantee only that the thread will be given preference over noncritical threads

hard real-time systems: Hard real-time systems have strict scheduling facilities - a thread must be serviced by its deadline and service after the deadline has expired is the same as no service at all.

event latency: The amount of time that elapses from when an event occurs to when it is serviced.

Interrupt latency: The period of time from the arrival of an interrupt at the CPU to the start of the routine that services the interrupt.

dispatch latency: The amount of time the dispatcher takes to stop one thread and put another thread onto CPU.

conflict phase: During scheduling, the time the dispatcher spends moving a thread off of a CPU and releasing resources held but lower-priority threads that are needed by the higher-priority thread that is about to be put onto CPU.

periodic: A type of real-time process that repeatedly moves between two modes at fixed intervals- needing CPU time and not needing CPU time.

rate: A periodic real-time process has a scheduling rate of $1 / p$ (where p is the length of its running period).

admission-control: In real-time scheduling, the scheduler may not allow a process to start if its scheduling request is impossible - if it cannot guarantee that the task will be serviced by its deadline.

rate-monotonic: The rate-monotonic scheduling algorithm schedules periodic tasks using a static priority policy with preemption.

Earliest-Deadline-First (EDF): A real-time scheduling algorithm in which the scheduler dynamically assigns priorities according to completion deadlines.

proportional share: Proportional share schedulers operate by allocating T shares among all applications assuring each gets a specific portion of CPU time.

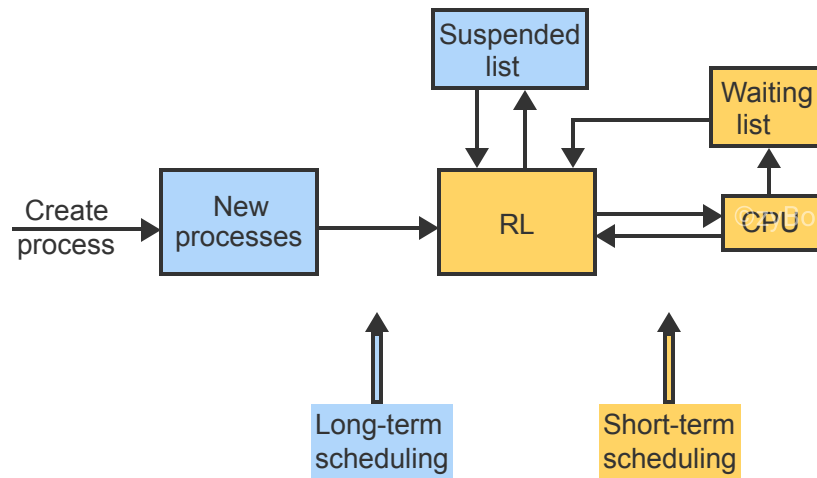
4.7 Principles of scheduling

Long-term vs short-term scheduling

Scheduling decisions are made at two different levels.

- **Long-term scheduling** decides when a process should enter the ready state and start competing for the CPU.
- **Short-term scheduling** decides which of the ready processes should run next on the CPU.





Animation content:

Static figure: A flowchart diagram for the process state transitions in a computer system during scheduling. An arrow labeled Create Process points to box named New Processes. The Box New processes points to box Ready List (RL) and the transition is labeled Long term scheduling. A double headed arrow points between RL and a box called CPU. CPU points to a box Waiting list, which points to RL. A double headed arrow points between RL and a box Suspended list. The transitions between CPU, RL, and Waiting list are labeled short-term scheduling.

Step 1: Processes are created at unpredictable times and enter the system with the process state: new. Box New processes appears with incoming Create process arrow.

Step 2: Long-term scheduling decides which processes are moved to the ready list (RL) to compete for the CPU.

Box RL and arrow from New processes to RL appears. Transition is marked as Long-term scheduling,

Step 3: Processes are also subject to long-term scheduling when suspended and later reactivated by the OS. Box Suspended list with arrows to and from RL appears.

Step 4: Short-term scheduling moves processes between the ready list and the CPU.

Box CPU with arrows to and from RL appear. Transitions are marked as Short-term scheduling.

Step 5: When a running process becomes blocked and later reenters the RL, short-term scheduling is again applied to let the process compete for the CPU.

Box Waiting list appears along with arrows from CPU and to RL.

Animation captions:

1. Processes are created at unpredictable times and enter the system with the process state: new.
2. Long-term scheduling decides which processes are moved to the ready list (RL) to compete for the CPU.
3. Processes are also subject to long-term scheduling when suspended and later reactivated by the OS.
4. Short-term scheduling moves processes between the ready list and the CPU.
5. When a running process becomes blocked and later reenters the RL, short-term scheduling is again applied to let the process compete for the CPU.

PARTICIPATION ACTIVITY

4.7.2: Scheduling frequency.



Long-term scheduling takes place _____
short-term scheduling.



- ☐ more frequently than
- ☐ less frequently than
- ☐ equally frequently as

PARTICIPATION ACTIVITY

4.7.3: Long-term vs short-term scheduling.



A process can move to the RL from different locations.

1) A process entering the RL from a list of
new processes is subject to _____.



- ☐ long-term scheduling
- ☐ short-term scheduling

2) A process entering the RL from the
suspended list is subject to _____.



- ☐ long-term scheduling
- ☐ short-term scheduling

3) A process entering the RL from the
waiting list is subject to _____.



- ☐ long-term scheduling
- ☐ short-term scheduling

4) A process entering the RL from the CPU
is subject to _____.

- ☐ long-term scheduling
- ☐ short-term scheduling

Preemptive vs non-preemptive scheduling

A **non-preemptive** scheduling algorithm allows a running process to continue until the process terminates or blocks on a resource.

A **preemptive** scheduling algorithm may stop the currently running process and choose another process to run. The decision is made whenever:

- A new process enters the ready list.
- A previously blocked or suspended process re-enters the RL.
- The OS periodically interrupts the currently running process to give other processes a chance to run.

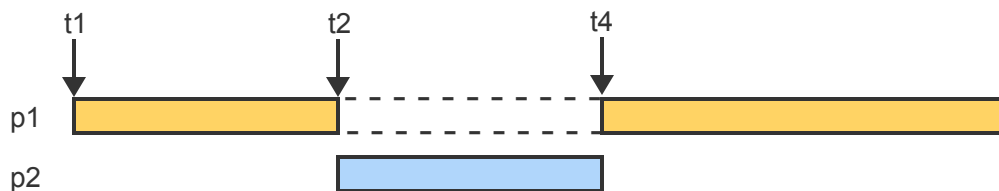
PARTICIPATION ACTIVITY

4.7.4: Non-preemptive vs preemptive scheduling.

Non-preemptive scheduling



Preemptive scheduling



Animation content:

Static figure: Two processes, p1 and p2, are shown as horizontal bars in both non-preemptive and preemptive scheduling. The non-preemptive case has time markers t1, t2, and t3 along the bars in various places. The preemptive case has time markers t1, t2, and t4. The bars have different lengths

and are either solid or dashed according to the status of the processes at the different times.

Step 1: Process p1 arrives at time t1 and starts executing. The bar for process p1 in the non-preemptive case appears solid between t1 and t2.

Step 2: A second process p2 arrives at time t2.

Step 3: Under a non-preemptive algorithm, p1 continues while p2 waits.

In the non-preemptive case, the p1 bar is extended until time t3, while p2 is represented by a dashed, empty bar that extends until p1 finishes, indicating that p2 is waiting.

Step 4: p2 starts when p1 terminates at time t3.

At time t3 where p1 terminates, the bar for p1 ends and the bar for p2 becomes solid, indicating that p2 starts executing.

Step 5: Under a preemptive algorithm, p1 stops at time t2.

The bar for p1 extends from t1 to t2 as in the non-preemptive case.

Step 6: If p2 is chosen to run next, p1 needs to wait until time t4 to continue.

The preemptive case shows that p2's bar is solid between t2 and t4 while p1's bar becomes dashed indicating that p1 is waiting while p2 runs. At t4, p1's bar becomes solid until p1 terminates.

Animation captions:

1. Process p1 arrives at time t1 and starts executing.
2. A second process p2 arrives at time t2.
3. Under a non-preemptive algorithm, p1 continues while p2 waits.
4. p2 starts when p1 terminates at time t3.
5. Under a preemptive algorithm, p1 stops at time t2.
6. If p2 is chosen to run next, p1 needs to wait until time t4 to continue.

PARTICIPATION ACTIVITY

4.7.5: Scheduling decisions under non-preemptive scheduling.



A non-preemptive algorithm makes a scheduling decision whenever ____.

1) a new process arrives in the RL



- ☐ True
- ☐ False

2) a process changes from the blocked state to the ready state

- ☐ True
☐ False

3) a process requests an unavailable resource

- ☐ True
☐ False

4) a process terminates

- ☐ True
☐ False

**PARTICIPATION
ACTIVITY**

4.7.6: Scheduling decisions under preemptive scheduling.

A preemptive algorithm makes a scheduling decision whenever ____.

1) a new process arrives in the RL

- ☐ True
☐ False

2) a process changes from the blocked state to the ready state

- ☐ True
☐ False

3) a process requests an unavailable resource

- ☐ True
☐ False

4) a process terminates

- ☐ True
☐ False

Priority for short-term scheduling

The **priority of a process** (or thread) is a numerical value that indicates the importance of the process relative to other processes. The scheduler uses this value to decide which process to run next.

The priority can be a constant value assigned when the process is created or can change dynamically based on some combination of the following parameters.

The **arbitration rule** decides which process should proceed if two or more processes have the same priority.

Table 4.7.1: Common parameters used to compute short-term priority.

Parameter	Explanation
Arrival	The point in time when the process enters the RL.
Departure	The point in time when the process leaves the RL by entering the blocked or suspended state, or by terminating all work.
Attained CPU time	The amount of CPU time used by the process since arrival.
Real time in system	The amount of actual time the process has spent in the system since arrival.
Total CPU time	The amount of CPU time the process will consume between arrival and departure. For short-term scheduling, total CPU time is sometimes called the CPU burst.
External priority	A numeric priority value assigned to the process explicitly at the time of creation.
Deadline	A point in time by which the work of the process must be completed.
Period	A time interval during which a periodically repeating computation must be completed. The end of each period is the implicit deadline for the current computation.
Other considerations	The resource requirements of a process, such as the amount of memory used, or the current load on the system.

For a process in the ready state, the _____ keeps increasing.

- ☐ arrival time
- ☐ attained CPU time
- ☐ real time in system
- ☐ total CPU time

**PARTICIPATION
ACTIVITY**

4.7.8: The meaning of the priority parameters.

1) A weather forecasting application is an example of a system with a deadline.

- ☐ True
- ☐ False

2) A video streaming application is an example of a periodic process.

- ☐ True
- ☐ False

3) The external priority is recorded in a process's PCB.

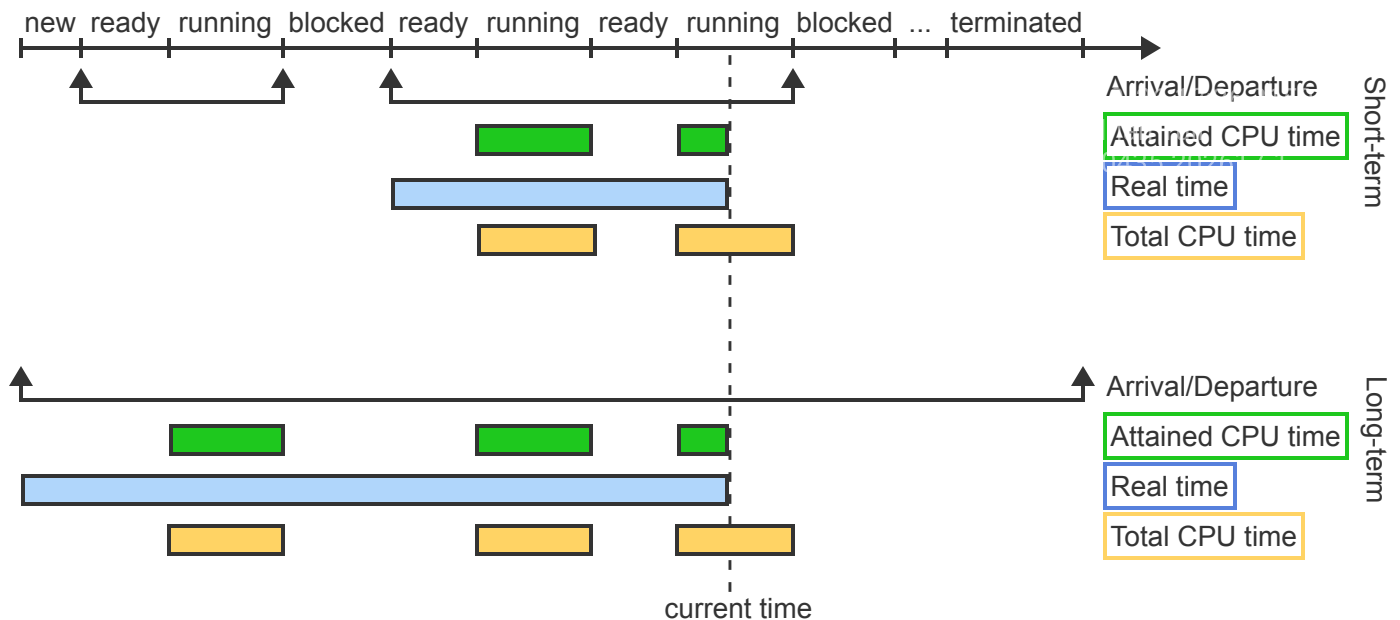
- ☐ True
- ☐ False

Priority for long-term scheduling

Long-term scheduling, which decides when a new process should enter the RL, also uses the concept of priority. Long-term priority can be based on the same parameters as short-term priority: arrival, departure, attained CPU time, real time in the system, total CPU time, and external priority. However, long-term scheduling occurs much less frequently than short-term scheduling and thus the decisions are made at a higher granularity of time.

- Arrival is the time of process creation.
- Departure is the time of process destruction.

Consequently, the attained CPU time and the total CPU time have different meanings in short-term and long-term scheduling.



Animation content:

Static figure: The images depict a horizontal timeline showing the states of a process with the different phases marked as new, ready, running, blocked, or terminated. Below, different brackets represent the arrival and departure periods of processes and Horizontal bars of various lengths represent periods of Attained CPU time, Real time, and Total CPU time. The top part of the timeline represents Short-term scheduling while the bottom part represents Long-term scheduling.

Step 1: A process goes through a series of state transitions starting from new and ending in terminated. The time line appears marked with the series of states new, ready, running, blocked, ready, running, ready, running, blocked, ..., terminated.

Step 2: For short-term scheduling, each arrival is a transition into ready state (entering RL) and each departure is a transition into blocked state (leaving RL).

In the case of short-term scheduling, each arrival of the process is marked at the beginning of a ready phase and the corresponding departure is marked at the start of a blocked phase. Thus a process experiences multiple arrival/departure events.

Step 3: For long-term scheduling, arrival is the creation of the process (entering new) and departure is the destruction of the process (leaving terminated).

In the case of long-term scheduling, the arrival of the process is marked at the beginning of the new

phase and the corresponding departure is marked at the end of the terminated phase. Thus a process experiences only one arrival and one departure event.

Step 4: The attained CPU time accumulates between arrival and departure while the process is running. A bar representing attained CPU time appears below the first running phase.

Step 5: For short-term scheduling, the attained CPU time restarts with each arrival and keeps accumulating until the current time.

A vertical bar labeled current time starts in the middle of the last running phase and cuts across the diagram. For short-term scheduling, attained CPU time covers the running phases since arrival and the first portion of the last running period up to the current time.

Step 6: For long-term scheduling, the attained CPU time keeps accumulating since the process's creation until the current time. For long-term scheduling, attained CPU time covers all previous running phases and the first portion of the last running phase up to the current time.

Step 7: The real time for short-term scheduling start with each arrival and accumulates until the current time regardless of the process's state (running or ready).

A bar representing the real time for short-term scheduling starts at the end of the blocked phase and ends at the current time.

Step 8: The real time for long-term scheduling start with the process creation and accumulates until the current time regardless of the process's state (running, ready, or blocked).

A bar representing the real time for long-term scheduling extends all the way from the new phase to the current time.

Step 9: The total CPU time is the accumulation of all CPU times (running state) from arrival to departure. For short-term scheduling the accumulation restarts with each new arrival.

Two bars representing the total CPU time for short-term scheduling cover the two running phases between arrival and departure.

Step 10: Long-term scheduling has only a single arrival. Thus the total CPU time is the accumulation of all CPU times (running states) from process arrival to process termination.

Three bars representing the total CPU time for long-term scheduling cover all running phases between arrival and departure.

Animation captions:

1. A process goes through a series of state transitions starting from new and ending in terminated.
2. For short-term scheduling, each arrival is a transition into ready state (entering RL) and each departure is a transition into blocked state (leaving RL).
3. For long-term scheduling, arrival is the creation of the process (entering new) and departure is the destruction of the process (leaving terminated).

4. The attained CPU time accumulates between arrival and departure while the process is running.
5. For short-term scheduling, the attained CPU time restarts with each arrival and keeps accumulating until the current time.
6. For long-term scheduling, the attained CPU time keeps accumulating since the process's creation until the current time.
7. The real time for short-term scheduling starts with each arrival and accumulates until the current time regardless of the process's state (running or ready).
8. The real time for long-term scheduling starts with the process creation and accumulates until the current time regardless of the process's state (running, ready, or blocked).
9. The total CPU time is the accumulation of all CPU times (running state) from arrival to departure. For short-term scheduling the accumulation restarts with each new arrival.
10. Long-term scheduling has only a single arrival. Thus the total CPU time is the accumulation of all CPU times (running states) from process arrival to process termination.

**PARTICIPATION
ACTIVITY**

4.7.10: Arrival/departure vs creation/destruction.



Arrival/departure times correspond to process creation/destruction in _____ scheduling.



- ☐ short-term
- ☐ long-term
- ☐ both short and long-term

**PARTICIPATION
ACTIVITY**

4.7.11: Attained and total CPU times under long-term and short-term scheduling.



A process p starts running at time 0, becomes blocked at time 2, and starts running again at time 5.

- 1) p 's attained CPU time under short-term scheduling at time 6 will be _____ time units.



Check

[Show answer](#)

- 2) p's attained CPU time under long-term scheduling at time 6 will be _____ time units.



Check

Show answer

- 3) If p terminates at time 7, the total CPU time under long-term scheduling will be _____ time units.



Check

Show answer

4.8 Requirements for efficient memory management

Logical vs physical memory

A computer's **physical memory (RAM)** is a hardware structure consisting of a linear sequence of words that hold a program during execution. A **word** is a fixed-size unit of data. A typical word size is 1, 2, or 4 bytes.

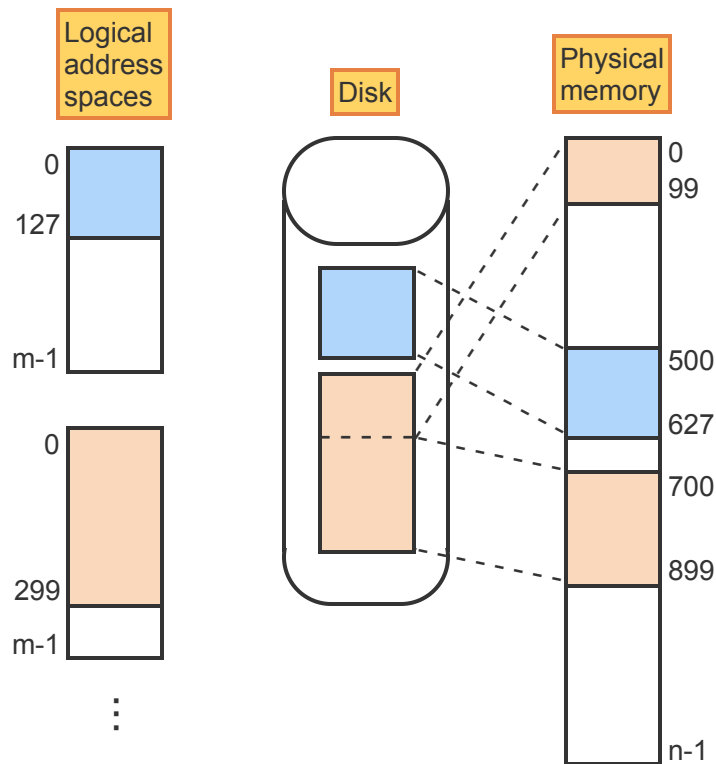
A **physical address** is an integer in the range $[0 : n-1]$ that identifies a word in a physical memory of size n . The address comprises a fixed number of bits. A memory of size n requires an address size of k bits, where $n = 2^k$.

During program development, the starting address of a program in physical memory is unknown. To facilitate program development and the sharing of physical memory, the concept of a logical address space is employed.

A **logical address space** is an abstraction of physical memory, consisting of a sequence of imaginary memory locations in a range $[0 : m-1]$, where m is the size of the logical address space.

A **logical address** is an integer in the range $[0 : m-1]$ that identifies a word in a logical address space. Prior to execution, a logical address space is mapped to a portion of physical memory and the program is copied into the corresponding locations.





Animation content:

Static figure: Elements that represent the concept of physical vs logical memory in a computer system. Column 1 shows two boxes representing logical address spaces labeled from top to bottom with address range $[0 : m-1]$. Column 2 shows a Disk that holds copies of some portion of physical memory, which is shown in column 3. The Physical memory has address labels in the range $[0 : n-1]$.

Step 1: A computer system has a single physical memory consisting of n words addressed by fixed-size physical addresses in the range $[0 : n-1]$. Physical memory appears.

Step 2: To facilitate program development and program loading, multiple logical address spaces are provided with possibly different address ranges $[0 : m-1]$. The two logical address spaces appear.

Step 3: The occupied portion of each logical address space is kept on disk. The portion $[0:127]$ of address space 1 and the portion $[0:299]$ of address space 2 are shown on the disk.

Step 4: A program may be copied to a contiguous region of physical memory when needed for execution. Logical space $[0 : 127]$ is copied to memory locations $[500 : 627]$.

Step 5: A program can also be copied to multiple disjoint regions of physical memory. The space [0 : 299] is mapped to the regions [0 : 99] and [700 : 899].

Animation captions:

1. A computer system has a single physical memory consisting of n words addressed by fixed-size physical addresses in the range [0 : $n-1$].
2. To facilitate program development and program loading, multiple logical address spaces are provided with possibly different address ranges [0 : $m-1$].
3. The occupied portion of each logical address space is kept on disk
4. A program may be copied to a contiguous region of physical memory when needed for execution. Logical space [0 : 127] is copied to memory locations [500 : 627].
5. A program can also be copied to multiple disjoint regions of physical memory. The space [0 : 299] is mapped to the regions [0 : 99] and [700 : 899].

PARTICIPATION ACTIVITY

4.8.2: Size of logical address space.



If the size of a logical address space is much larger than the average program size then a lot of space is wasted ____.

1) in physical memory



- ☐ True
- ☐ False

2) on the disk



- ☐ True
- ☐ False

PARTICIPATION ACTIVITY

4.8.3: Address size.



With an address space of 1024 words, the address 6 will occupy ____ bits.



- ☐ 3
- ☐ 2^3
- ☐ 10



The word size of an address space depends on ____.



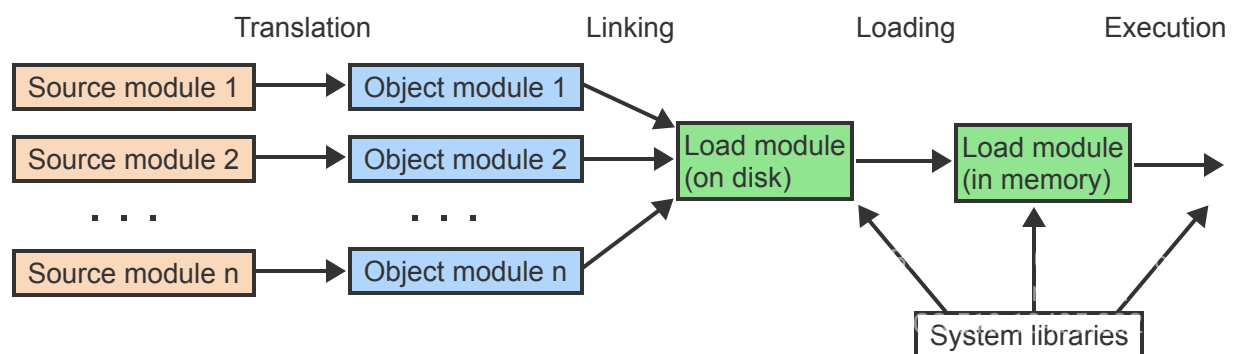
- ☐ the number of bits in the address
- ☐ the number of words in the address space
- ☐ neither the address size nor the address space size

Program transformations

A **source module** is a program or a program component written in a symbolic language, like C, or an assembly language, that must be translated by a compiler or assembler into executable machine code.

An **object module** is the machine-language output of a compiler or assembler generated from a source module. An object module may be self-contained and executable or multiple object modules may be linked together into a load module by a linker or linkage editor.

A **load module** is a program or combination of programs in a form ready to be loaded into main memory and executed.



Animation content:

Static figure: A flowchart of the stages of program transformation from source modules to execution. Source modules 1, 2, ..., n represent different source modules of the program. Translation arrows point from each source module to corresponding Object module 1, 2, ..., n.

A Linking arrow leads from the object modules to Load module (on disk). A Loading arrow leads to Load module (in memory), and finally, an arrow points to Execution. Arrows from System libraries point to the Load module (in memory).

Step 1: A program consists of one or more source modules written in a high-level language or assembly language. Source modules appear.

Step 2: Each module is translated separately into an object module using a compiler or assembler.

Object modules appear.

Step 3: Object modules are combined into a load module by a linker. The load module is kept on disk.

Step 4: Prior to execution, the load module is copied to physical memory by a loader.

Step 5: Various system libraries, in the form of object modules, may be included in the load module by the linker, the loader, or during execution.

Animation captions:

1. A program consists of one or more source modules written in a high-level language or assembly language.
2. Each module is translated separately into an object module using a compiler or assembler.
3. Object modules are combined into a load module by a linker. The load module is kept on disk.
4. Prior to execution, the load module is copied to physical memory by a loader.
5. Various system libraries, in the form of object modules, may be included in the load module by the linker, the loader, or during execution.



1) A source module is the input to the ____.

- ☐ compiler/assembler
- ☐ linker
- ☐ loader

2) An object module is the input to the ____.

- ☐ compiler/assembler
- ☐ linker
- ☐ loader

3) A load module is the input to the ____.

- ☐ compiler/assembler
- ☐ linker
- ☐ loader

4) System libraries may be added to a ____.

- ☐ source module
- ☐ object module
- ☐ load module

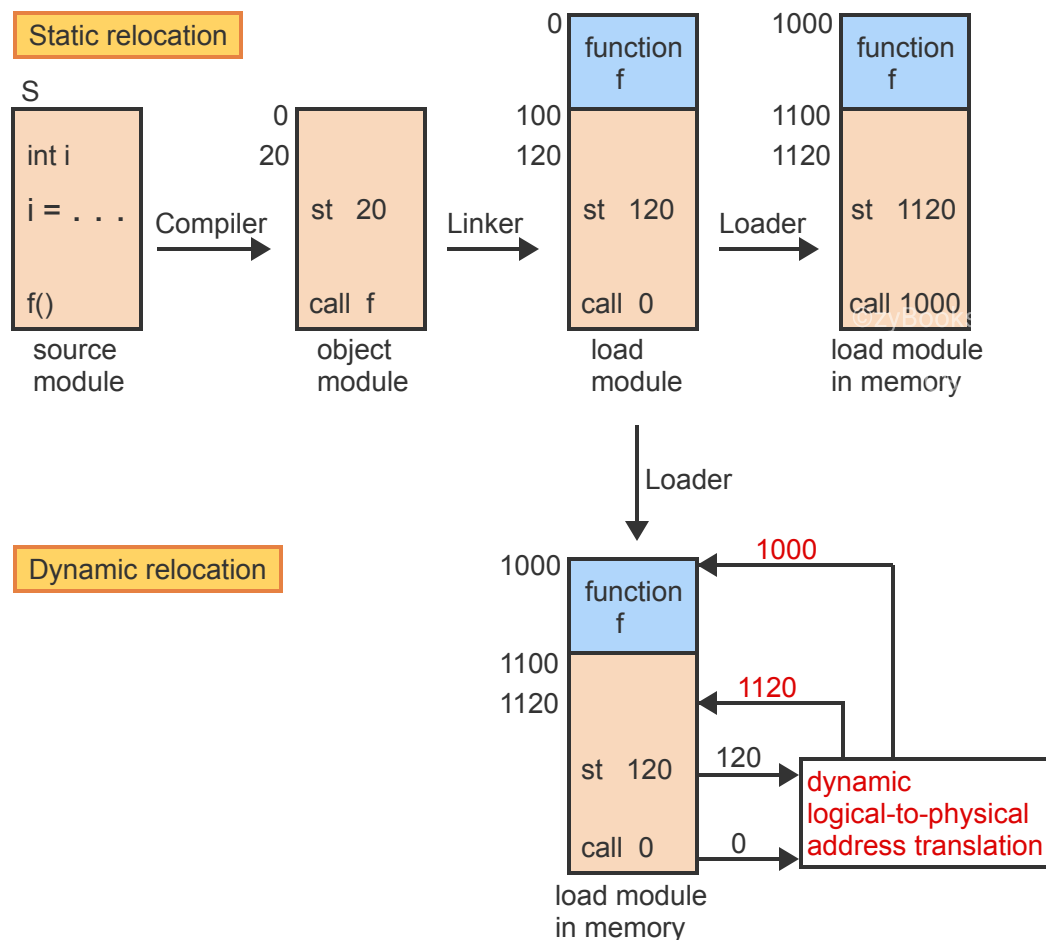
Relocation and address binding

Since the physical addresses of the program components are not known until load time, the compiler/assembler and the linker assume logical address spaces starting with address 0. During each step of the program transformation, the logical addresses of instructions and data may be changing. Only during the final step of program loading are all logical addresses bound to actual physical addresses in memory.

Program **relocation** is the act of moving a program component from one address space to another. The relocation may be between two logical address spaces or from a logical address space to a physical address space.

Static relocation binds all logical addresses to physical addresses prior to execution.

Dynamic relocation postpones the binding of a logical address to a physical address until the addressed item is accessed during execution.



Animation content:

Static figure: A series of transformations of a program from source module to object module to load module and finally to load module in memory is shown for both static and dynamic relocation.

Step 1: A program consists of multiple source modules, each written in a high-level language. Module S defines a variable `i`, which is later assigned a value. Function `f` resides in a different module. Static relocation. The source module S contains the pseudo code: `int i; i=...; f()`.

Step 2: The compiler assumes a logical address 0 and assigns `i` to address 20. The store instruction `st` then references location 20. The location of `f` is unknown and is kept as a symbolic external reference. The object module shows the addresses 0 and 20 and contains the pseudo code: `st 20; call f`.

Step 3: The linker also assumes a logical starting address of 0 and combines all object modules into one load module. S is relocated to reflect the starting address 100. The call now refers to the start of `f`. The load module starts at address 0 with the function `f`, followed by the object module starting at address 100. The pseudo code of the object module is now: `st 120; call 0`.

Step 4: With static relocation, the loader must adjust all logical addresses to physical addresses when copying the module into memory at the starting physical address of 1000. The function f starts at address 1000. The original module starts at address 1100 and contains the pseudo code: st 1120; call 1000.

Step 5: With dynamic relocation, the load module is copied into memory also starting at address 1000 but without any changes to the addresses in the code. The module contains the unmodified pseudo code: st 120; call 0.

Step 6: Relocation is done at runtime by adding the starting address 1000 to every address when referenced. A box labeled dynamic logical-to-physical translation appears. The address 0 and 120 are fed from the module into the box, which translates them into 1000 and 1120, respectively. The transformed addresses are used to access the load module. 1000 point to the start of function f and 1120 points into the original module.

Animation captions:

1. A program consists of multiple source modules, each written in a high-level language. Module S defines a variable i, which is later assigned a value. Function f resides in a different module.
2. The compiler assumes a logical address 0 and assigns i to address 20. The store instruction st then references location 20. The location of f is unknown and is kept as a symbolic external reference.
3. The linker also assumes a logical starting address of 0 and combines all object modules into one load module. S is relocated to reflect the starting address 100. The call now refers to the start of f.
4. With static relocation, the loader must adjust all logical addresses to physical addresses when copying the module into memory at the starting physical address of 1000.
5. With dynamic relocation, the load module is copied into memory also starting at address 1000 but without any changes to the addresses in the code.
6. Relocation is done at runtime by adding the starting address 1000 to every address when referenced.

PARTICIPATION ACTIVITY

4.8.8: Modified addresses under static and dynamic relocation.



The above load module is moved into memory starting at address 500.

1) The variable i will reside at physical address ____.



- ☐ 120
- ☐ 520
- ☐ 620

2) With static relocation, the operand of the instruction st will be ____.

- ☐ 120
- ☐ 600
- ☐ 620

3) With dynamic relocation, the operand of the instruction st will be ____.

- ☐ 120
- ☐ 520
- ☐ 620

Implementing dynamic relocation using relocation registers

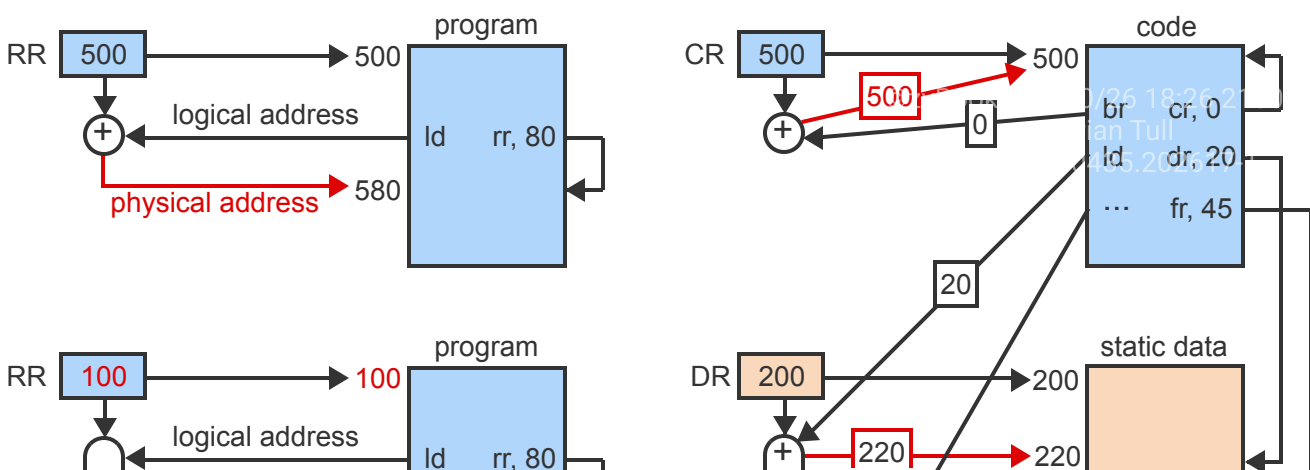
A **relocation register** contains the physical starting address of a program or program component in memory.

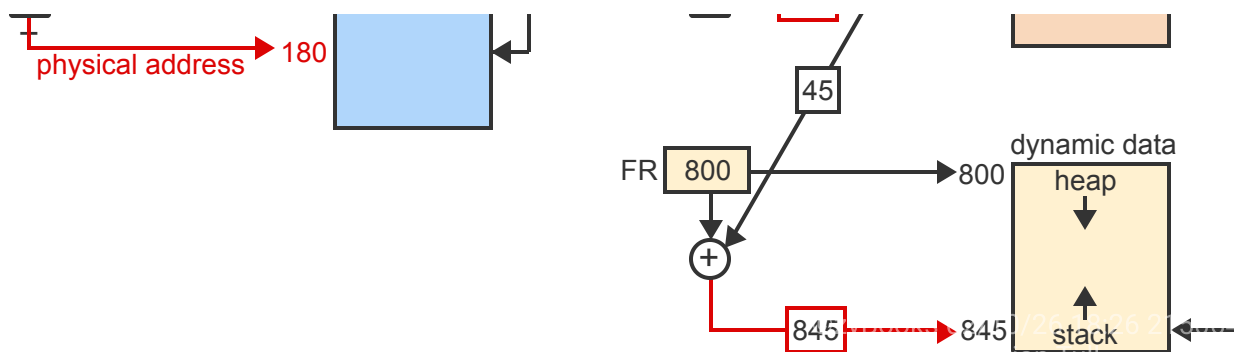
Most programs consist of 3 main components: code, static data, and dynamic data. A simple memory management scheme treats the 3 components as one unit, which must be moved into one contiguous area of memory. Dynamic relocation can be implemented using a single relocation register, which is loaded with the program's starting address. The register content is then added automatically by the CPU to every logical address of the program to generate the physical address.

A more flexible management scheme treats the 3 components as separate modules, each of which may reside in a different area of memory. Three relocation registers, each loaded with the starting address of one of the modules, then accomplish dynamic relocation by being added to all logical addresses at runtime.

PARTICIPATION ACTIVITY

4.8.9: Dynamic relocation using relocation registers.





Animation content:

Static figure: A representation of memory allocation and address translation for a single-module program and a multi-module program using relocation registers. The first is a single-module program initially at physical address 500 in memory, with a corresponding relocation register (RR) holding the same address. The second is the program moved to physical address 100, and RR updated accordingly. The next set is a multi-module program with separate modules for code, static data, and dynamic data. Each module has its own relocation register: code (CR) at physical address 500, static data (DR) at physical address 200, and dynamic data (FR) at physical address 800. Different instructions such as `ld` (load) and `br` (branch) specify which relocation register to use with flags indicating the type of data they access, either code, static, or dynamic.

Step 1: When a single-module program is loaded into memory, the starting address 500 is loaded into the relocation register RR as the base address for future address calculations.

Step 2: The load instruction `ld` uses a flag, `rr`, to indicate that the operand 80 is not an absolute value but a logical address referring to 580. The CPU adds 500 to 80, which results in the physical address 580. $RR=500$. An arrow points from the instruction "`ld rr,80`" to the physical address $500+80=580$.

Step 3: When the program is moved to a new area of memory starting at address 100, the code does not change. Only RR is loaded with 100, which guarantees that correct physical addresses are generated at runtime. $RR=100$. An arrow points from the instruction "`ld rr,80`" to the physical address $100+80=180$.

Step 4: When a program consists of separate modules, different relocation registers point to the code (CR), the static data (DR), and the dynamic data (FR).

Step 5: Instructions that refer to the code area (Ex: branch) use a flag (`cr`) to tell the CPU to add the content of CR to the logical address 0. Thus the physical address is $500+0=500$ and refers to the start of the code. $CR=500$. An arrow points from the instruction "`br cr,0`" to the physical address $500+0=500$ in the code segment.

Step 6: Instructions that refer to static data (Ex: load) use the flag `dr` to tell the CPU to add the content of `DR` to the logical address. Thus the load instruction refers to address 220 within static data. $DR=200$. An arrow points from the instruction "`ld dr,20`" in the code segment to the physical address $200+20=220$ in the static data segment.

Step 7: Similarly, instructions that refer to the heap or stack use the flag `fr` to tell the CPU to add the content of `FR` to the logical address and thus refer to addresses within dynamic data.

$FR=800$. An arrow points from the instruction "`... fr,45`" in the code segment to the physical address $800+45=845$ in the dynamic data segment.

Animation captions:

1. When a single-module program is loaded into memory, the starting address 500 is loaded into the relocation register `RR` as the base address for future address calculations.
2. The load instruction `ld` uses a flag, `rr`, to indicate that the operand 80 is not an absolute value but a logical address referring to 580. The CPU adds 500 to 80, which results in the physical address 580.
3. When the program is moved to a new area of memory starting at address 100, the code does not change. Only `RR` is loaded with 100, which guarantees that correct physical addresses are generated at runtime.
4. When a program consists of separate modules, different relocation registers point to the code (`CR`), the static data (`DR`), and the dynamic data (`FR`).
5. Instructions that refer to the code area (Ex: branch) use a flag (`cr`) to tell the CPU to add the content of `CR` to the logical address 0. Thus the physical address is $500+0=500$ and refers to the start of the code.
6. Instructions that refer to static data (Ex: load) use the flag `dr` to tell the CPU to add the content of `DR` to the logical address. Thus the load instruction refers to address 220 within static data.
7. Similarly, instructions that refer to the heap or stack use the flag `fr` to tell the CPU to add the content of `FR` to the logical address and thus refer to addresses within dynamic data.

PARTICIPATION ACTIVITY

4.8.10: Correct use of relocation flags.



Assume the code section in the above example contains the instructions:

1. Branch to a location within the code section
2. Load a value `x` from the static data section
3. Add a constant operand 100 to `x`
4. Store the new value `x+100` in the dynamic data section

1) Instruction 1 will use ____.

- ☐ flag cr
- ☐ flag dr
- ☐ flag fr
- ☐ no flag



2) Instruction 2 will use ____.

- ☐ flag cr
- ☐ flag dr
- ☐ flag fr
- ☐ no flag



3) Instruction 3 will use ____.

- ☐ flag cr
- ☐ flag dr
- ☐ flag fr
- ☐ no flag



4) Instruction 4 will use ____.

- ☐ flag cr
- ☐ flag dr
- ☐ flag fr
- ☐ no flag



Free space management

As programs are moved into and out of memory at runtime, the memory space becomes a checkerboard of free and occupied areas of different sizes. The OS keeps track of all free spaces, referred to as holes, using a linked list and must find a hole of appropriate size whenever a new program component is to be loaded into memory.

The OS must also coalesce any neighboring holes resulting from the removal of programs from memory to prevent the memory from becoming a fragmented collection of increasingly smaller holes.

Different search strategies have been explored:

First-fit always starts the search from the beginning of the list and allocates the first hole large enough to accommodate the request.

Next-fit starts each search at the point of the last allocation.

Best-fit searches the entire list and chooses the smallest hole large enough to accommodate the request.

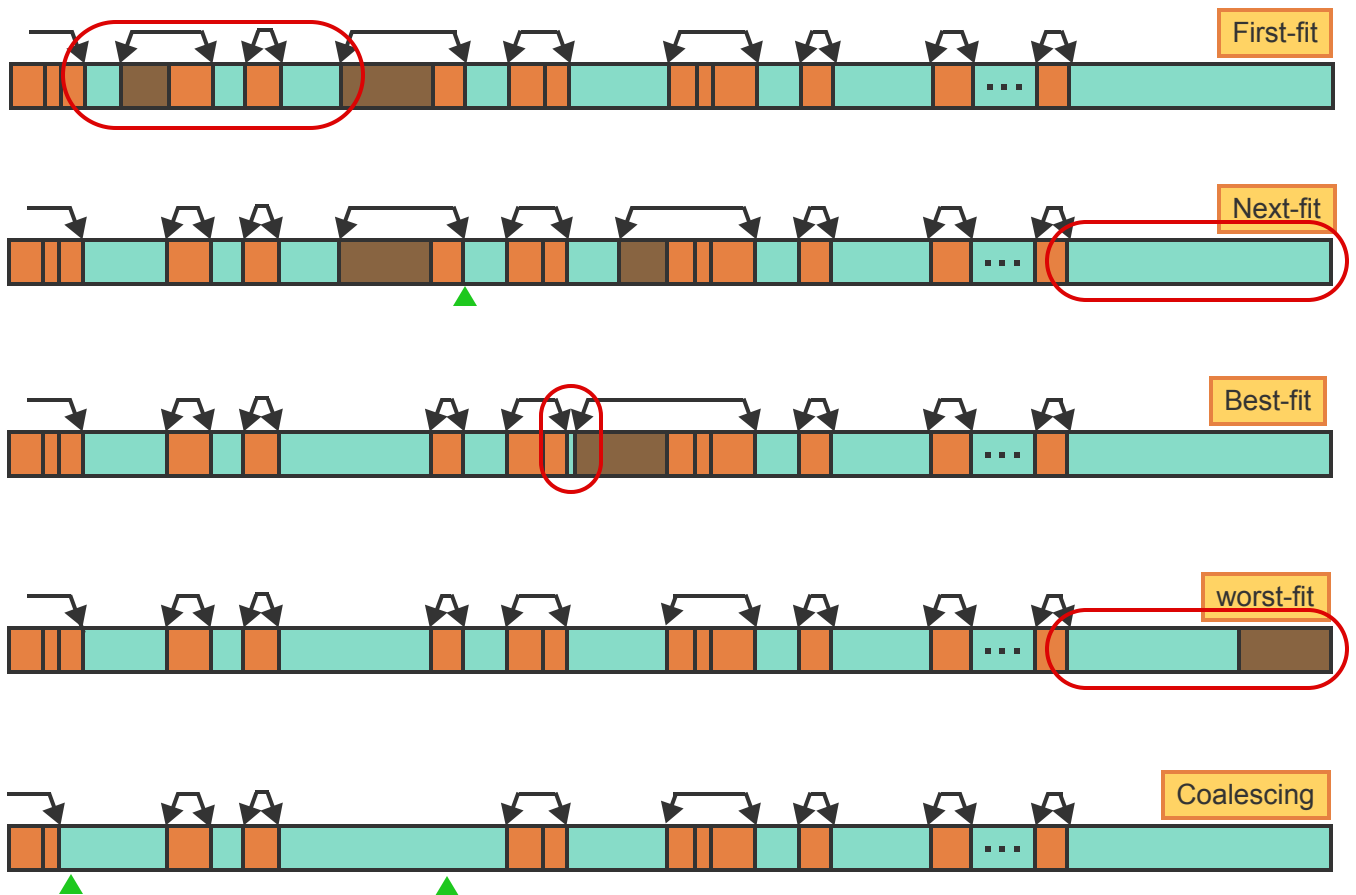
Worst-fit takes the opposite approach from best-fit by always choosing the largest available hole for any request.

Simulation of different strategies have not produced a clear winner. The choice depends on the average request size relative to the memory size, and the variance of request sizes.

**PARTICIPATION
ACTIVITY**

4.8.11: Allocation strategies and coalescing.

CS509: Operating Systems
Adrian Tull
CS-510-10435-2016-Fall



Animation content:

Static figure: A representation of memory space with a series of occupied blocks and holes. The blocks vary in size, indicating different amounts of free and occupied memory. Double-ended arrows, representing doubly linked lists, connect adjacent holes. The series is replicated for different strategies: First-fit, Next-fit, Best-fit, Worst-fit, and Coalescing.

Step 1: All holes are connected using a doubly linked list. Starting from the beginning, first-fit allocates the first hole large enough for the given request. First fit. A new block appears and keeps sliding from the start of the series using the forward links until a hole large enough is found. The block is placed at the end of the hole and the links are adjusted to point to the now smaller hole.

Step 2: Each new search starts from the beginning of the list. Consequently, many small holes accumulate over time at the beginning of the list and slow down the search.

Step 3: To distribute the holes more evenly, next-fit starts each search with (1) the remainder hole of the last allocation or (2) the sequentially next hole. Choice (2) is assumed here. A marker indicates the last hole visited. The search for the next hole starts from the marker.

Step 4: Next-fit, also called the circular first-fit, does not return to the start and thus cycles through all holes. The drawback is that large holes at the end decrease in size and may become unavailable for large requests.

Step 5: Best-fit searches all holes and selects the smallest one large enough for the request, which frequently leaves a small, unusable hole behind. The small holes increase search time and decrease utilization. Best fit. A new block appears and slides through all holes of the series before landing in the smallest possible hole.

Step 6: Worst-fit allocates space from the largest hole available. The negative effect is that large holes get broken up and become unavailable for large requests. Best fit. A new block appears and slides through all holes of the series before landing in the largest hole.

Step 7: In all cases, a released block must always be coalesced with any neighboring holes to prevent indefinite fragmentation of memory. An occupied block is released and the linked list is adjusted to reflect the increased size of the adjacent hole.

Animation captions:

1. All holes are connected using a doubly linked list. Starting from the beginning, first-fit allocates the first hole large enough for the given request.
2. Each new search starts from the beginning of the list. Consequently, many small holes accumulate over time at the beginning of the list and slow down the search.
3. To distribute the holes more evenly, next-fit starts each search with (1) the remainder hole of the last allocation or (2) the sequentially next hole. Choice (2) is assumed here.
4. Next-fit, also called the circular first-fit, does not return to the start and thus cycles through all holes. The drawback is that large holes at the end decrease in size and may become unavailable for large requests.
5. Best-fit searches all holes and selects the smallest one large enough for the request, which frequently leaves a small, unusable hole behind. The small holes increase search time and

decrease utilization.

6. Worst-fit allocates space from the largest hole available. The negative effect is that large holes get broken up and become unavailable for large requests.
7. In all cases, a released block must always be coalesced with any neighboring holes to prevent indefinite fragmentation of memory.

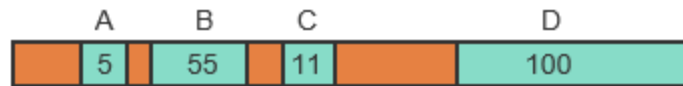
**PARTICIPATION
ACTIVITY**

4.8.12: Allocations using different strategies.

StudyBooks 02/10/2018 26:21:50 adrian.tull



The list contains 4 holes, A through D, with the given sizes.



1) First-fit will place a request for 10 bytes into hole ____.



- ☐ A
- ☐ B
- ☐ C
- ☐ D

2) Best-fit will place a request for 10 bytes into hole ____.



- ☐ A
- ☐ B
- ☐ C
- ☐ D

3) Worst-fit will place a request for 10 bytes into hole ____.



- ☐ A
- ☐ B
- ☐ C
- ☐ D

4) If the last request was placed into hole D, then (assuming the search starts with the next hole) next-fit will place a subsequent request for 20 bytes into hole ____.

- ☐ A
- ☐ B
- ☐ C
- ☐ D

**PARTICIPATION
ACTIVITY**

4.8.13: Comparison of strategies.

1) Which strategy tends to preserve large holes?

- ☐ Next-fit
- ☐ Best-fit
- ☐ Worst-fit

2) Which strategy always needs to search all holes?

- ☐ First-fit
- ☐ Next-fit
- ☐ Best-fit

**CHALLENGE
ACTIVITY**

4.8.1: Requirements for efficient memory management.

726670.4301284.qx3zqy7

Start

Memory contains 6 holes with their sizes shown in the table.

Hole	A	B	C	D	E	F
Size	110	120	60	130	70	100

A sequence of requests for 3 blocks have arrived with sizes 10, 90, and 80. Select the hole allocated to each request by the first-fit allocation strategy.

Block with size 10:

Block with size 90: 
Block with size 80: 

Enter the number of holes that first-fit had to examine in order to allocate the 3 blocks.

Ex: 2



Check

Next



EXERCISE

4.8.1: Static and dynamic relocation.



The animation in this section entitled "Static vs dynamic relocation" shows a load module in memory under static and dynamic relocation.

An additional function occupying 50 memory words is to be linked in front of function f.

The load module is then loaded into memory starting at location 1500.

- (a) Show how the addresses change in the load module
1. prior to loading
 2. after loading under static relocation
 3. after loading under dynamic relocation



EXERCISE

4.8.2: Allocating a portion of a hole.



- (a) A hole of size m is used to satisfy a request for n bytes of memory, where $m > n$. The holes are organized as a linked list. Is allocating the last n bytes of the hole better than allocating the first n bytes? Why or why not?



EXERCISE

4.8.3: Searching overhead in allocation.



- (a) Memory contains m holes. To satisfy a request for 1 byte of memory, how many holes need to be visited under the different allocation strategies of first-fit, next-fit, best-fit, and worst-fit?

**EXERCISE**

4.8.4: Comparison of allocation strategies.



Memory contains 3 holes of 10 MB each. A sequence of 14 requests for 1 MB each is to be processed.

- (a) For each of the four memory allocation strategies, determine the sizes of the remaining holes after all 14 requests have been satisfied.

**EXERCISE**

4.8.5: Comparison of allocation strategies.



Memory contains 6 holes with the sizes: 190, 550, 220, 420, 650, and 110.

A sequence of requests for 4 block is to be satisfied: A = 210, B = 430, C = 100, and D = 420.

- (a) Determine which holes are allocated to which request by each of the 4 allocation schemes.

**EXERCISE**

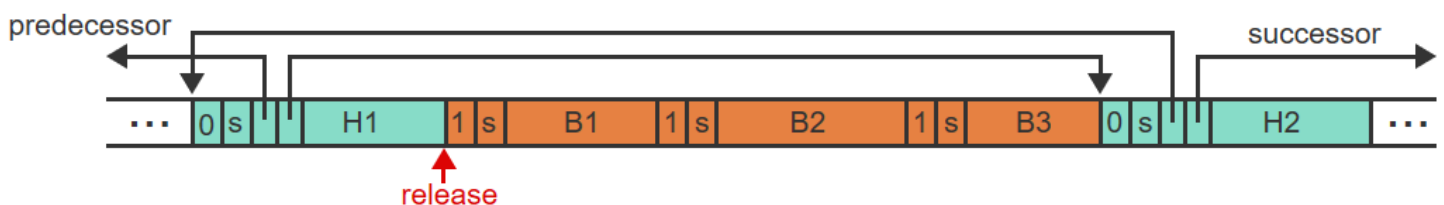
4.8.6: Representing allocated blocks and holes.



Main memory consists of allocated blocks and holes of various sizes.

- Each allocated block starts with a 1, followed by the block size, s.
- Each hole starts with a 0, followed by the hole size, s.
- Holes are linked together using a doubly linked list. Since space in the holes is unused, the links are kept inside the holes.

Example:

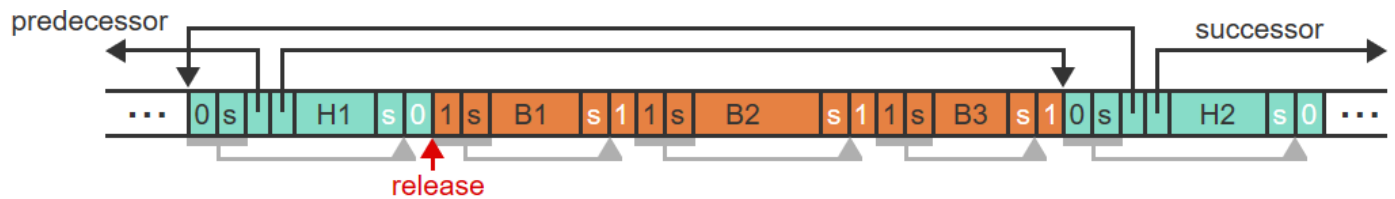


- The memory contains 3 contiguous blocks (B1, B2, B3) of different sizes.
- H1 and H2 are holes linked together with successor and predecessor pointers.
- Block B1 is to be released

- (a) When a block is released, the memory manager must check if the left and/or the right neighbor is a hole, in which case the newly created hole must be coalesced with the existing hole. Describe the steps the memory manager must take to check if the right neighbor of block B1 is a hole.

- (b) Describe the steps the memory manager must take to check if the left neighbor of block B1 is a hole.

- (c) The checking of the left neighbor can significantly be simplified if the sizes and the 0/1 tags are replicated at the end of each occupied block and each hole:



Describe the steps the memory manager must take to check if the left neighbor of block B1 is a hole.

- (d) Why must the size fields be replicated at the end of each hole/block and not just the 0/1 tag?